# Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge

Yiping Kang Johann Hauswald Cao Gao Austin Rovinski
Trevor Mudge Jason Mars Lingjia Tang
Clarity Lab
University of Michigan - Ann Arbor, MI, USA
{ypkang, jahausw, caogao, rovinski, tnm, profmars, lingjia}@umich.edu

## Abstract

The computation for today's intelligent personal assistants such as Apple Siri, Google Now, and Microsoft Cortana, is performed in the cloud. This cloud-only approach requires significant amounts of data to be sent to the cloud over the wireless network and puts significant computational pressure on the datacenter. However, as the computational resources in mobile devices become more powerful and energy efficient, questions arise as to whether this cloud-only processing is desirable moving forward, and what are the implications of pushing some or all of this compute to the mobile devices on the edge.

In this paper, we examine the status quo approach of cloud-only processing and investigate computation partitioning strategies that effectively leverage both the cycles in the cloud and on the mobile device to achieve low latency, low energy consumption, and high datacenter throughput for this class of intelligent applications. Our study uses 8 intelligent applications spanning computer vision, speech, and natural language domains, all employing state-of-the-art Deep Neural Networks (DNNs) as the core machine learning technique. We find that given the characteristics of DNN algorithms, a fine-grained, layer-level computation partitioning strategy based on the data and computation variations of each layer within a DNN has significant latency and energy advantages over the status quo approach.

Using this insight, we design `Neurosurgeon`, a lightweight scheduler to automatically partition DNN computation between mobile devices and datacenters at the granularity of neural network layers. `Neurosurgeon` does not require per-application profiling. It adapts to various DNN architectures, hardware platforms, wireless networks, and server load levels, intelligently partitioning computation for best latency or best mobile energy. We evaluate `Neurosurgeon` on a state-of-the-art mobile development platform and show that it improves end-to-end latency by 3.1× on average and up to 40.7×, reduces mobile energy consumption by 59.5% on average and up to 94.7%, and improves datacenter throughput by 1.5× on average and up to 6.7×.

***Keywords*** mobile computing; cloud computing; deep neural networks; intelligent applications



*ASPLOS '17,* April 08-12, 2017, Xi'an, China



DOI: http://dx.doi.org/10.1145/3037697.3037698

## 1. Introduction

The way we interact with today's mobile devices is rapidly changing as these devices are increasingly personal and knowledgeable. Intelligent Personal Assistants (IPAs), such as Apple Siri, Google Now, and Microsoft Cortana, are integrated by default on mobile devices and are expected to grow in popularity as wearables and smart home devices continue to gain traction [1, 2]. The primary interface with these intelligent mobile applications is using speech or images to navigate the device and ask questions. Demand for this mode of interaction is expected to replace the traditional text based inputs [3–5].

Processing speech and image inputs for IPA applications requires accurate and highly sophisticated machine learning techniques, the most common of which are Deep Neural Networks (DNNs). DNNs have become increasingly popular as the core machine learning technique in these applications due to their ability to achieve high accuracy for tasks such as speech recognition, image classification and natural language understanding. Many companies, including Google, Microsoft, Facebook, and Baidu, are using DNNs as the machine learning component for numerous applications in their production systems [6–8].

Prior work has shown that speech or image queries for DNN-based intelligent applications require orders of magnitude more processing than text based inputs [9]. The common wisdom has been that traditional mobile devices cannot support this large amount of computation with reasonable latency and energy consumption. Thus, the status quo approach used by web service providers for intelligent applications has been to host all the computation on high-end cloud servers [10–13]. Queries generated from a user's mo-

# 神经外科医生：云和移动边缘之间的协作智能

Yiping Kang Johann Hauswald Cao Gau Austin Rovinski Trevor Mudge Jason Mars Lingjia Tang Clarity Lab 密歇根大学 - 美国密歇根州安娜堡

{ypkang, jahausw, caogao, rovinski, tnm, profmars, lingjia}@umich.edu

## 抽象的

当今的智能个人助理（例如 Apple Siri、Google Now 和 Microsoft Cortana）的计算是在云端执行的。这种仅基于云的方法需要通过无线网络将大量数据发送到云，并给数据中心带来巨大的计算压力。然而，随着移动设备中的计算资源变得更加强大和节能，人们提出了这样的问题：这种纯云处理是否值得继续发展，以及将部分或全部计算推到边缘移动设备会产生什么影响。

在本文中，我们研究了仅云处理的现状方法，并研究了有效利用云中和移动设备上的周期的计算分区策略，以实现此类智能应用程序的低延迟、低能耗和高数据中心吞吐量。我们的研究使用了 8 个跨越计算机视觉、语音和自然语言领域的智能应用程序，所有应用程序都采用最先进的深度神经网络 (DNN) 作为核心机器学习技术。我们发现，考虑到 DNN 算法的特点，基于 DNN 中每一层的数据和计算变化的细粒度、层级计算划分策略比现有方法具有显着的延迟和能量优势。

利用这一见解，我们设计了 Neurosurgeon，这是一个轻量级调度程序，可以按照神经网络层的粒度自动在移动设备和数据中心之间划分 DNN 计算。Neurosurgeon 不需要每个应用程序分析。它适应各种 DNN架构、硬件平台、无线网络和服务器负载水平，智能划分计算最佳延迟或最佳移动能源。我们在最先进的移动开发平台上对 Neurosur- geon 进行了评估，结果表明，它使端到端延迟平均提高了 3.1×，最高可达 40.7×，移动能耗平均降低了 59.5%，最高可达 94.7%，数据中心吞吐量平均提高了 1.5×，最高可达 6.7×。

***Keywords*** 移动计算；云计算；深度神经网络；智能应用

## 一、简介

随着当今移动设备变得越来越个性化和知识化，我们与这些设备的交互方式正在迅速变化。智能个人助理 (IPA)，例如 Apple Siri、Google Now 和 Microsoft Cortana，默认集成在移动设备上，并且随着可穿戴设备和智能家居设备的不断发展，预计将越来越受欢迎 [1, 2]。这些智能移动应用程序的主要界面是使用语音或图像来导航设备并提出问题。对这种交互模式的需求预计将取代传统的基于文本的输入[3-5]。

处理 IPA 应用的语音和图像输入需要准确且高度复杂的机器学习技术，其中最常见的是深度神经网络 (DNN)。 DNN 作为这些应用中的核心机器学习技术越来越受欢迎，因为它们能够实现语音识别、图像分类和自然语言理解等任务的高精度。许多公司，包括谷歌、微软、Facebook 和百度，都在其生产系统中使用 DNN 作为众多应用程序的机器学习组件 [6-8]。

先前的工作表明，基于 DNN 的智能应用程序的语音或图像查询需要比基于文本的输入多几个数量级的处理 [9]。人们普遍认为，传统移动设备无法以合理的延迟和能耗支持如此大量的计算。因此，网络服务提供商用于智能应用程序的现状方法是将所有计算托管在高端云服务器上[10-13]。从用户的移动中生成的查询



*ASPLOS '17,* April 08-12, 2017, Xi'an, China

© 2017 ACM. ISBN 978-1-4503-4465-4/17/04…$15.00

DOI: http://dx.doi.org/10.1145/3037697.3037698

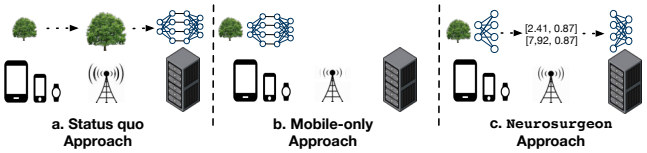



**Figure 1: Status quo, mobile-only and the Neurosurgeon approach. Status quo approach performs all computation remotely in the cloud, the mobile-only approach performs all computation locally on the mobile device, and the Neurosurgeon approach partitions computation between the cloud and mobile device.**

bile device are sent to the cloud for processing, as shown in Figure 1a. However, with this approach, large amounts of data (e.g., images, video and audio) are uploaded to the server via the wireless network, resulting in high latency and energy costs.

While data transfer becomes the latency and energy bottleneck, performance and energy efficiency of modern mobile hardware have continued to improve through powerful mobile SoC integration [14, 15]. Motivated by this observation, this work re-examines the computation breakdown for intelligent applications between mobile and cloud. In particular, we investigate how computation can be pushed out of the cloud and onto the mobile devices on the edge to execute all or parts of these conventionally cloud-only applications. Key questions we address in this work include:

**1.** How feasible it is to execute large-scale intelligent workloads on today's mobile platforms?

**2.** At what point is the cost of transferring speech and image data over the wireless network too high to justify cloud processing?

**3.** What role should the mobile edge play in providing processing support for intelligent applications requiring heavy computation?

Based on our investigation using 8 DNN-based intelligent applications spanning the domains of vision, speech, and natural language, we discover that, for some applications, due to the high data transfer overhead, locally executing on the mobile device (Figure 1b) can be up to 11× faster than the cloud-only approach (Figure 1a). Furthermore, we find that instead of limiting the computation to be either executed entirely in the cloud or entirely on the mobile, a fine-grained layer-level partitioning strategy based on a DNN's topology and constituent layers can achieve far superior end-to-end latency performance and mobile energy efficiency. By pushing compute out of the cloud and onto the mobile devices, we also improve datacenter throughput, allowing a given datacenter to support many more user queries, and creating a win-win situation for both the mobile and cloud systems.

Given the observation that ideal fine-grained DNN partition points depend on the layer compositions of the DNN, the particular mobile platform used, the wireless network configuration and the server load, we design a lightweight dynamic scheduler, Neurosurgeon. Neurosurgeon is a runtime system spanning cloud and mobile platforms that automatically identifies the ideal partition points in DNNs and orchestrates the distribution of computation between the mobile device and the datacenter. As Figure 1c shows, Neurosurgeon partitions the DNN computation and takes advantage of the processing power of both the mobile and the cloud while reducing data transfer overhead. The detailed contributions of this paper are as follows:

- **In-depth examination of the status quo –** We show the latency and energy consumption of executing state-of-the-art DNNs in the cloud and on the mobile device. We observe that uploading via the wireless network is the bottleneck of the status quo approach, and mobile execution often provides better latency and energy consumption than the status quo approach. (Section 3)
- **DNN compute and data size characteristics study –** We provide an in-depth layer-level characterization of the compute and data size of 8 DNNs spanning across computer vision, speech and natural language processing. Our investigation reveals that DNN layers have significantly different compute and data size characteristics depending on their type and configurations. (Section 4)
- **DNN computation partitioning across the cloud and mobile edge –** Based on the compute and data characterization of DNN layers, we show that partitioning DNN at layer granularity offers significant performance benefits. We then design a systematic approach to identify the optimal points to partition computation for reduced latency and mobile energy consumption across a suite of applications. (Section 4)
- **Neurosurgeon runtime system and layer performance prediction models –** We develop a set of models to predict the latency and power consumption of a DNN layer based on its type and configuration, and create Neurosurgeon, a system to intelligently partition DNN computation between the mobile and cloud. We demonstrate that Neurosurgeon significantly improves end-to-end latency, reduces mobile energy consumption, and improves datacenter throughput. (Sections 5 and 6)

Our evaluation on a suite of 8 DNN applications shows that using Neurosurgeon on average improves end-to-end latency by 3.1×, reduces mobile energy consumption by 59.5%, and improves datacenter throughput by 1.5×.

## 2. Background

In this section, we provide an overview of Deep Neural Network (DNN) and describe how computer vision, speech, and natural language processing applications leverage DNNs as their core machine learning algorithm.

DNNs are organized in a directed graph where each node is a processing element (a neuron) that applies a function to its input and generates an output. Figure 2 depicts a 5 layer

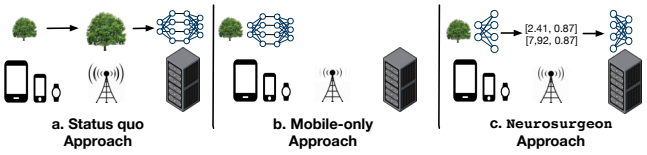



图 1：现状、仅限移动设备和 Neurosurgeon 方法。现状方法在云中远程执行所有计算，仅移动方法在移动设备上本地执行所有计算，而 Neurosurgeon方法在云和移动设备之间划分计算。

胆设备发送到云端进行处理，如图1a所示。然而，这种方法需要通过无线网络将大量数据（例如图像、视频和音频）上传到服务器，从而导致较高的延迟和能源成本。

虽然数据传输成为延迟和能源瓶颈，但现代移动硬件的性能和能源效率通过强大的移动 SoC 集成不断提高[14, 15]。受这一观察的启发，这项工作重新审视了移动和云之间智能应用程序的计算分解。特别是，我们研究了如何将计算从云中推出并转移到边缘的移动设备上，以执行这些传统上仅限云的应用程序的全部或部分。我们在这项工作中解决的关键问题包括：

1. 在当今的移动平台上执行大规模智能工作负载的可行性如何?

2. 在什么情况下，通过无线网络传输语音和图像数据的成本过高，以至于无法证明云处理的合理性?

3. 移动边缘在为需要大量计算的智能应用提供处理支持方面应发挥什么作用?

根据我们对涵盖视觉、语音和自然语言领域的 8 个基于 DNN 的智能应用程序的调查，我们发现，对于某些应用程序，由于数据传输开销较高，在移动设备上本地执行（图 1b）可以比仅云方法（图 1a）快 11×。此外，我们发现，基于 DNN 拓扑和组成层的细粒度层级划分策略可以实现远远优越的端到端延迟性能和移动能源效率，而不是限制计算完全在云端或完全在移动设备上执行。通过将计算从云端推到移动设备上，我们还提高了数据中心吞吐量，允许给定的数据中心支持更多的用户查询，并为移动和云系统创造双赢的局面。

鉴于理想的细粒度 DNN 划分点取决于 DNN 的层组成、使用的特定移动平台、无线网络配置和服务器负载，我们设计了一个轻量级动态调度程序 Neurosurgeon。 Neurosurgeon 是一个跨越云和移动平台的运行时系统自动识别 DNN 中的理想划分点并协调移动设备和数据中心之间的计算分布。如图 1c 所示，Neu- rosurgeon 对 DNN 计算进行分区，并利用移动设备和云的处理能力，同时减少数据传输开销。本文的详细贡献如下：

- 深入研究现状——我们展示了在云端和移动设备上执行最先进的 DNN 的延迟和能耗。我们观察到，通过无线网络上传是现状方法的瓶颈，而移动执行通常比现状方法提供更好的延迟和能耗。（第 3 节）

- DNN 计算和数据大小特征研究——我们提供了跨计算机视觉、语音和自然语言处理的 8 个 DNN 的计算和数据大小的深入分层特征。我们的调查表明，DN N 层根据其类型和配置具有显着不同的计算和数据大小特征。（第 4 节）

- 跨云和移动边缘的 DNN 计算分区——基于 DNN 层的计算和数据特征，我们表明在层粒度上分区 DNN 可提供显着的性能优势。然后，我们设计了一种系统方法来确定分区计算的最佳点，以减少一系列应用程序的延迟和移动能耗。（第 4 节）

- Neurosurgeon 运行时系统和层性能预测模型——我们开发了一组模型来根据 DNN 层的类型和配置来预测 DNN 层的延迟和功耗，并创建 Neuro- surgeon，一个在移动设备和云端之间智能划分 DNN 计算的系统。我们证明 Neurosurgeon 显着改善了端到端延迟，降低了移动能耗，并提高了数据中心吞吐量。（第 5 节和第 6 节）

我们对一套 8 个 DNN 应用程序的评估表明，使用 Neurosurgeon 平均可将端到端延迟提高 3.1×，将移动能耗降低 59.5%，并将数据中心吞吐量提高 1.5×。

## 2. 背景

在本节中，我们将概述深度神经网络 (DNN)，并描述计算机视觉、语音和自然语言处理应用程序如何利用 DN N 作为其核心机器学习算法。

DNN 以有向图的形式组织，其中每个节点都是一个处理元素（神经元），它将函数应用于其输入并生成输出。图2描绘了5层

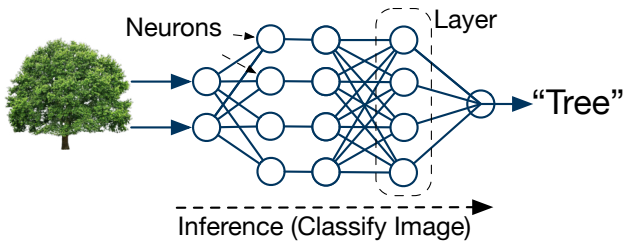



**Figure 2: A 5-layer Deep Neural Network (DNN) classifies input image into one of the pre-defined classes.**

DNN for image classification where computation flows from left to right. The edges of the graph are the connections between each neuron defining the flow of data. Multiple neurons applying the same function to different parts of the input define a layer. For a forward pass through a DNN, the output of a layer is the input to the next layer. The depth of a DNN is determined by the number of layers. Computer Vision (CV) applications use DNNs to extract features from an input image and classify the image into one of the predefined classes. Automatic Speech Recognition (ASR) applications use DNNs to generate predictions for speech feature vectors, which will then be post-processed to produce the most-likely text transcript. Natural Language Processing (NLP) applications use DNNs to analyze and extract semantic and syntactic information from word embedding vectors generated from input text.

## 3. Cloud-only Processing: The Status Quo

Currently, the status quo approach used by cloud providers for intelligent applications is to perform all DNN processing in the cloud [10–13]. A large overhead of this approach is in sending data over the wireless network. In this section, we investigate the feasibility of executing large DNNs entirely on a state-of-the-art mobile device, and compare with the status quo.

### 3.1 Experimental setup

We use a real hardware platform, representative of today's state-of-the-art mobile devices, the Jetson TK1 mobile platform developed by NVIDIA [16] and used in the Nexus 9 tablet [17]. The Jetson TK1 is equipped with one of NVIDIA's latest mobile SoC, Tegra K1: a quad-core ARM A15 and a Kepler mobile GPU with a single streaming multiprocessor (Table 1).

**Table 1: Mobile Platform Specifications**

| Hardware | Specifications |
|---|---|
| System | Tegra K1 SoC |
| CPU | 4-Plus-1 quad-core ARM Cortex A15 CPU |
| Memory | 2 GB DDR3L 933MHz |
| GPU | NVIDIA Kepler with 192 CUDA Cores |

**Table 2: Server Platform Specifications**

| Hardware | Specifications |
|---|---|
| System | 4U Intel Dual CPU Chassis, 8 × PCIe 3.0 × 16 slots |
| CPU | 2 × Intel Xeon E5-2620 V2, 6C, 2.10 GHz |
| HDD | 1TB 2.5" HDD |
| Memory | 16 × 16GB DDR3 1866MHz ECC/Server Memory |
| GPU | NVIDIA Tesla K40 M-Class 12 GB PCIe |

Our server platform is equipped with an NVIDIA Tesla K40 GPU, one of NVIDIA's latest offering in server class GPUs (Table 2).

We use Caffe [18], an actively developed open-source deep learning library, for the mobile and server platform. For the mobile CPU, we use OpenBLAS [19], a NEON-vectorized matrix multiplication library and use the 4 cores available. For both GPUs, we use cuDNN [20], an optimized NVIDIA library that accelerates key layers in Caffe, and use Caffe's CUDA implementations for rest of the layers.

### 3.2 Examining the Mobile Edge

We investigate the capability of the mobile platform to execute a traditionally cloud-only DNN workload. We use AlexNet [21] as our application, a state-of-the-art Convolutional Neural Network for image classification. Prior work has noted that AlexNet is representative of today's DNNs deployed in server environments [22].

In Figure 3, we break down the latency of an AlexNet query, a single inference on a 152KB image. For wireless communication, we measure the bandwidth of 3G, LTE, and Wi-Fi on several mobile devices using TestMyNet [23].

**Communication Latency –** Figure 3a shows the latency to upload the input image via 3G, LTE, and Wi-Fi. The slowest is 3G connection taking over 870ms. LTE and Wi-Fi connection require 180ms and 95ms to upload, respectively, showing that the network type is critical for achieving low latency for the status quo approach.

**Computation Latency –** Figure 3b shows the computation latency on mobile CPU, GPU and cloud GPU. The slowest platform is the mobile CPU taking 382ms to process while the mobile GPU and cloud GPU take 81ms and 6ms, respectively. Note that the mobile CPU's time to process the image is still 2.3× faster than uploading input via 3G.

**End-to-end Latency –** Figure 3c shows the total latency required by the status quo and the mobile-only approach. Annotated on top of each bar is the fraction of the end-to-end latency spent on computation. The status quo approach spends less than 6% of the time computing on the server and over 94% of the time transferring data. The mobile GPU achieves a lower end-to-end latency than the status quo approach using LTE and 3G, while the status quo approach using LTE and Wi-Fi performs better than mobile CPU execution.

**Energy Consumption –** We measure the energy consumption of the mobile device using a Watts Up? meter [24] and

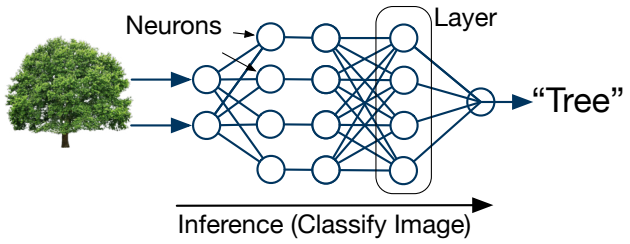



图 2：5 层深度神经网络 (DNN) 将输入图像分类为预定义的类别之一。

用于图像分类的 DNN，其中计算从左到右流动。图的边是定义数据流的每个神经元之间的连接。将相同功能应用于输入的不同部分的多个神经元定义了一个层。对于通过 DNN 的前向传递，一层的输出是下一层的输入。 DNN 的深度由层数决定。计算机视觉 (CV) 应用程序使用 DNN 从输入图像中提取特征，并将图像分类到预定义的类别之一。自动语音识别（ASR）应用程序使用 DNN 生成语音特征向量的预测，然后对其进行后处理以生成最可能的文本转录。自然语言处理 (NLP) 应用程序使用 DNN 从输入文本生成的词嵌入向量中分析和提取语义和句法信息。

## 3.纯云处理：现状

目前，云提供商用于智能应用程序的现状方法是在云中执行所有 DNN 处理[10-13]。这种方法的一大开销是通过无线网络发送数据。在本节中，我们将研究完全在最先进的移动设备上执行大型 DNN 的可行性，并与现状进行比较。

### 3.1 实验装置

我们使用真正的硬件平台，代表当今最先进的移动设备，即由 NVIDIA [16] 开发并用于 Nexus 9 平板电脑 [17] 的 Jetson TK1 移动平台。 Jetson TK1 配备了 NVIDIA 最新的移动 SoC 之一 Tegra K1：四核 ARM A15 和带有单个流式多处理器的 Kepler 移动 GPU（表 1）。

表 1：移动平台规格

| Hardware | Specifications |
|---|---|
| System | Tegra K1 SoC |
| CPU | 4-Plus-1 quad-core ARM Cortex A15 CPU |
| Memory | 2 GB DDR3L 933MHz |
| GPU | NVIDIA Kepler with 192 CUDA Cores |

表 2：服务器平台规格

| Hardware | Specifications |
|---|---|
| System | 4U Intel Dual CPU Chassis, 8×PCIe 3.0×16 slots |
| CPU | 2× Intel Xeon E5-2620 V2, 6C, 2.10 GHz |
| HDD | 1TB 2.5” HDD |
| Memory | 16× 16GB DDR3 1866MHz ECC/Server Memory |
| GPU | NVIDIA Tesla K40 M-Class 12 GB PCIe |

我们的服务器平台配备了 NVIDIA Tesla K40 GPU，这是 NVIDIA 最新的服务器级 GPU 产品之一（表 2）。

我们使用 Caffe [18]，一个积极开发的开源深度学习库，用于移动和服务器平台。对于移动 CPU，我们使用 OpenBLAS [19]，这是一个 NEON 向量化矩阵乘法库，并使用 4 个可用核心。对于这两个 GPU，我们使用 cuDNN [20]，这是一个优化的 NVIDIA 库，可加速 Caffe 中的关键层，并使用 Caffe 的 CUDA 实现来处理其余层。

### 3.2 检查移动边缘

我们研究了移动平台执行传统的仅云 DNN 工作负载的能力。我们使用 AlexNet [21] 作为我们的应用程序，这是一种用于图像分类的最先进的卷积神经网络。之前的工作指出 AlexNet 是当今服务器环境中部署的 DNN 的代表 [22]。

在图 3 中，我们分解了 AlexNet 查询的延迟，即对 152KB 图像的单个推理。对于无线通信，我们使用 Test MyNet [23] 测量多个移动设备上的 3G、LTE 和 Wi-Fi 带宽。

通信延迟 - 图 3a 显示通过 3G、LTE 和 Wi-Fi 上传输入图像的延迟。最慢的是 3G 连接，需要 870 毫秒以上。 LTE 和 Wi-Fi 连接分别需要 180 毫秒和 95 毫秒上传，这表明网络类型对于实现现有方法的低延迟至关重要。

计算延迟——图 3b 显示了移动 CPU、GPU 和云 GPU 上的计算延迟。最慢的平台是移动 CPU，需要 382 毫秒进行处理，而移动 GPU 和云 GPU 分别需要 81 毫秒和 6 毫秒。请注意，移动 CPU 处理图像的时间仍然比通过 3G 上传输入快 2.3×。

端到端延迟——图 3c 显示了现状和仅移动方法所需的总延迟。每个条形顶部注释的是用于计算的端到端延迟的比例。现状方法仅花费不到 6% 的时间在服务器上进行计算，而超过 94% 的时间用于传输数据。与使用 LTE 和 3G 的现状方法相比，移动 GPU 实现了更低的端到端延迟，而使用 LTE 和 Wi-Fi 的现状方法的性能优于移动 CPU 执行。

能源消耗 – 我们使用 Watts Up? 测量移动设备的能源消耗？ 米 [24] 和

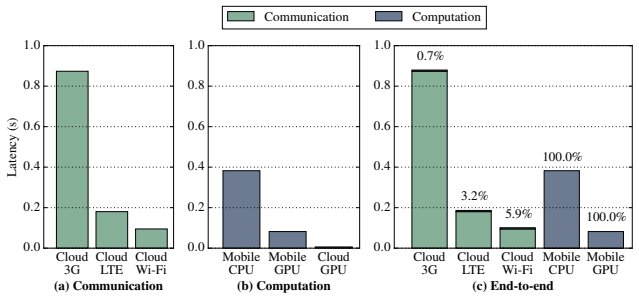



**Figure 3: Latency breakdown for AlexNet (image classification). The cloud-only approach is often slower than mobile execution due to the high data transfer overhead.**

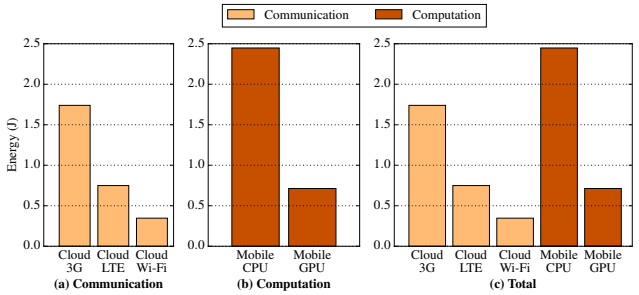



**Figure 4: Mobile energy breakdown for AlexNet (image classification). Mobile device consumes more energy transferring data via LTE and 3G than computing locally on the GPU.**

techniques described by Huang et al. [25]. Similar to the trends shown in Figure 3a, Figure 4a shows that the communication energy is heavily dependent on the type of wireless network used. In Figure 4b, the mobile device's energy consumption is higher on the CPU than the GPU (while the GPU needs more power, the device is used for a shorter burst thus it consumes less total energy). Figure 4c shows the total mobile energy consumption for the cloud-only approach and mobile execution where the energy in the cloud-only approach is dominated by communication. The mobile GPU consumes less energy than transferring input via LTE or 3G for cloud processing, while cloud processing via Wi-Fi consumes less energy than mobile execution.

***Key Observations*** – 1) The data transfer latency is often higher than mobile computation latency, especially on 3G and LTE. 2) Cloud processing has a significant computational advantage over mobile processing, but it does not always translate to end-to-end latency/energy advantage due to the dominating data transfer overhead. 3) Local mobile execution often leads to lower latency and energy consumption than the cloud-only approach, while the cloud-only approach achieves better performance if using fast Wi-Fi connection.

# 4. Fine-grained Computation Partitioning

Based on the findings in Section 3, the question arises as to whether it is advantageous to partition DNN computation between the mobile device and cloud. Based on the observation that DNN layers provide an abstraction suitable for partitioning computation, we begin with an analysis of the data and computation characteristics of state-of-the-art DNN architectures at the layer granularity.

## 4.1 Layer Taxonomy

Before the layer-level analysis, it is important to understand the various types of layers present in today's DNNs.

**Fully-connected Layer (`fc`) –** All the neurons in a fully-connected layer are exhaustively connected to all the neurons in the previous layer. The layer computes the weighted sum of the inputs using a set of learned weights.

**Convolution & Local Layer (`conv`, `local`) –** Convolution and local layers convolve the image with a set of learned filters to produce a set of feature maps. These layers mainly differ in the dimensions of their input feature maps, the number and size of their filters, and the stride with which the filters are being applied.

**Pooling Layer (`pool`) –** Pooling layers apply a pre-defined function (e.g., max or average) over regions of input feature maps to group features together. These layers mainly differ in the dimension of their input, size of the pooling region, and the stride with which the pooling is applied.

**Activation Layer –** Activation layers apply a non-linear function to each of its input data individually, producing the same amount of data as output. Activation layers present in the neural networks studied in this work include sigmoid layer (`sig`), rectified-linear layer (`relu`), and hard Tanh layer (`htanh`).

Other layers studied in this work include: **normalization layer (`norm`)** normalizes features across spatially grouped feature maps; **softmax layer (`softmax`)** produces a probability distribution over the number of possible classes for classification; **argmax layer (`argmax`)** chooses the class with the highest probability; and **dropout layer (`dropout`)** randomly ignores neurons during training to avoid model over-fitting and are passed through during prediction.

## 4.2 Characterizing Layers in AlexNet

We first investigate the data and computation characteristics of each layer in AlexNet. These characteristics provide insights to identify a better computation partitioning between mobile and cloud at the layer level. In the remainder of this and subsequent sections, we use the GPU in both mobile and server platforms.

**Per-layer Latency –** The left bars (light-colored) in Figure 5 show the latency of each layer on the mobile platform, arranged from left to right in their sequential execution order. The convolution (`conv`) and fully-connected layers (`fc`) are the most time-consuming layers, representing over 90% of the total execution time. Convolution layers in the middle (`conv3` and `conv4`) takes longer to execute than the early convolution layers (`conv1` and `conv2`). Larger number of filters are applied by the convolution layers later in the DNN to progressively extract more robust and representative fea-

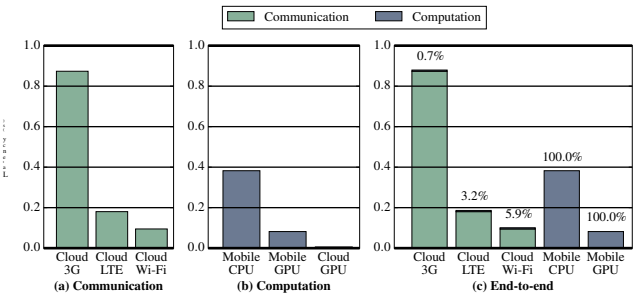



图 3：AlexNet 的延迟细分（图像分类）。由于数据传输开销较高，纯云方法通常比移动执行速度慢。

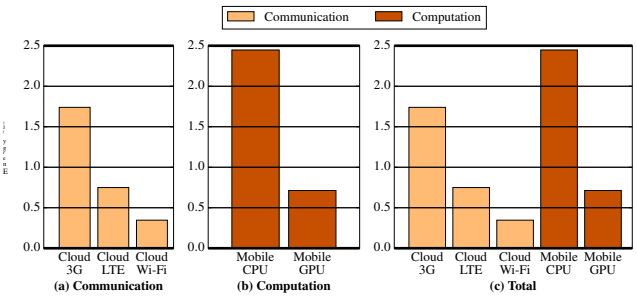



图 4：AlexNet 的移动能源细分（图像分类）。移动设备通过 LTE 和 3G 传输数据比在 GPU 上本地计算消耗更多的能量。

黄等人描述的技术。 [25]。与图 3a 中显示的趋势类似，图 4a 显示通信能量在很大程度上取决于所使用的无线网络的类型。在图 4b 中，移动设备的 CPU 能耗高于 GPU（虽然 GPU 需要更多电量，但设备使用的突发时间较短，因此消耗的总能耗较少）。图 4c 显示了纯云方法和移动执行的总移动能耗，其中纯云方法中的能量主要由通信决定。与通过 LTE 或 3G 传输输入进行云处理相比，移动 GPU 消耗的能源更少，而通过 Wi-Fi 进行的云处理比移动执行消耗的能源更少。

***Key Observations*** – 1) 数据传输延迟通常高于移动计算延迟，尤其是在 3G 和 LTE 上。 2）云处理比移动处理具有显着的计算优势，但由于数据传输开销占主导地位，它并不总是转化为端到端延迟/能源优势。 3）本地移动执行通常会比纯云方法带来更低的延迟和能耗，而如果使用快速 Wi-Fi 连接，纯云方法可以实现更好的性能。

## 4. 细粒度计算划分

根据第 3 节的发现，出现了在移动设备和云之间划分 DNN 计算是否有利的问题。基于对 DNN 层提供适合分区计算的抽象的观察，我们首先分析最先进的 DNN 架构在层粒度上的数据和计算特征。

### 4.1 层分类

在进行层级分析之前，了解当今 DNN 中存在的各种类型的层非常重要。

全连接层（`fc`）——全连接层中的所有神经元都详尽地连接到前一层中的所有神经元。该层使用一组学习的权重计算输入的加权和。

卷积和局部层（`conv`，`local`） - 卷积和局部层将图像与一组学习过滤器进行卷积，以生成一组特征图。这些层的主要区别在于输入特征图的尺寸、滤波器的数量和大小以及应用滤波器的步幅。

池化层 (`pool`) – 池化层在输入特征图的区域上应用预定义的函数（例如最大值或平均值），以将特征分组在一起。这些层的主要区别在于输入的维度、池化区域的大小以及应用池化的步幅。

激活层——激活层将非线性函数分别应用于每个输入数据，产生与输出相同数量的数据。这项工作中研究的神经网络中存在的激活层包括 sigmoid 层 (`sig`)、整流线性层 (`relu`) 和硬 Tanh 层 (`htanh`)。

这项工作中研究的其他层包括： 归一化层（`norm`）对空间分组特征图上的特征进行归一化； softmax 层（`softmax`）生成用于分类的可能类别数量的概率分布；argmax层(`argmax`)选择概率最高的类；丢失层（`dropout`）在训练期间随机忽略神经元以避免模型过度拟合，并在预测期间通过。

### 4.2 AlexNet 中的特征层

我们首先研究 AlexNet 中每一层的数据和计算特征。这些特征为在层级别确定移动和云之间更好的计算划分提供了见解。在本节的其余部分和后续部分中，我们在移动平台和服务器平台中使用 GPU。

每层延迟 - 图 5 中的左侧条形图（浅色）显示移动平台上每一层的延迟，按顺序执行顺序从左到右排列。卷积层 (`conv`) 和全连接层 (`fc`) 是最耗时的层，占总执行时间的 90% 以上。中间的卷积层（`conv3` 和 `conv4`）比早期的卷积层（`conv1` 和 `conv2`）需要更长的执行时间。DNN 中的卷积层稍后会应用更多数量的滤波器，以逐步提取更稳健和更具代表性的特征。

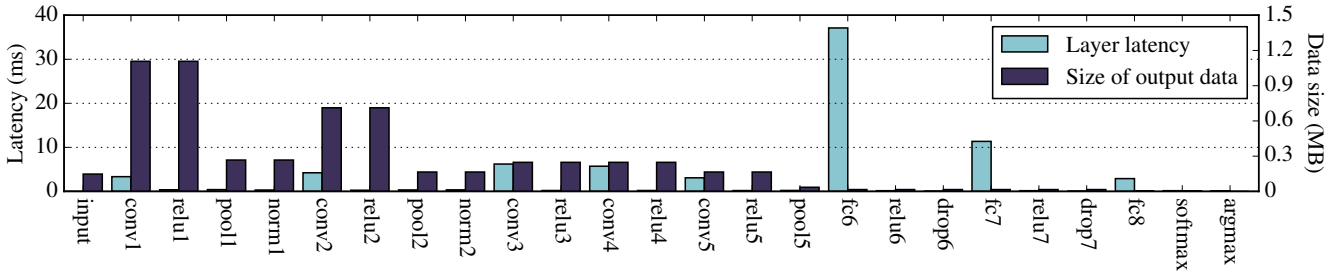
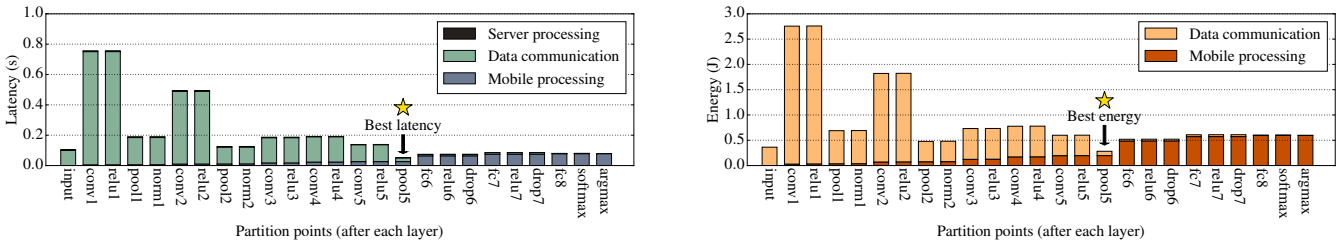



**Figure 5: The per layer execution time (the light-colored left bar) and size of data (the dark-colored right bar) after each layer's execution (input for next layer) in AlexNet. Data size sharply increases then decreases while computation generally increases through the network's execution.**




**(a) AlexNet latency** **(b) AlexNet energy consumption**

**Figure 6: End-to-end latency and mobile energy consumption when choosing different partition points. After the execution of every layer is considered a partition point. Each bar represents the total latency (a) or mobile energy (b) if the DNN is partitioned after the layer marked on the X-axis. The left-most bar represents cloud-only processing and the right-most bar represents mobile-only processing. The partition points for best latency and mobile energy are annotated.**

tures, increasing the amount of computation. On the other hand, fully-connected layers are up to one magnitude slower than the convolution layers in the network. The most time-consuming layer is the layer `fc6`, a fully-connected layer deep in the DNN, taking 45% of the total execution time.

**Data Size Variations –** The right bars (dark-colored) in Figure 5 shows the size of each layer's output data, which is also the input to the next layer. The first three convolution layers (`conv1`, `conv2` and `conv3`) generate large amounts of output data (shown as the largest dark bars) as they apply hundreds of filters over their input feature maps to extract interesting features. The data size stays constant through the activation layers (`relu1 - relu5`). The pooling layers sharply reduce the data size by up to 4.7× as they summarize regions of neighboring features by taking the maximum. The fully-connected layers deeper in the network (`fc6 - fc8`) gradually reduce the data size until the softmax layer (`softmax`) and argmax layer (`argmax`) at the end reduce the data to be one classification label.

***Key Observations –*** 1) Depending on its type and location in the network, each layer has a different computation and data profile. 2) The latency of convolution and pooling layers on the mobile GPU are relatively small, while fully-connected layers incur high latency. 3) Convolution and pooling layers are mostly at the front-end of the network, while fully-connected layers are at the back-end. 4) With convolution layers increasing data and then pooling layers reducing data, the front-end layers altogether reduce the size of data gradually. Data size in the last few layers are smaller than the original input. 5) The findings that data size is generally decreasing at the front-end, and per-layer mobile latency is generally higher at the back-end, indicates the unique opportunity for computation partitioning in the middle of the DNN between the mobile and cloud.

### 4.3 Layer-granularity Computation Partitioning

The analysis in Section 4.2 indicates that there exist interesting points within a neural network to partition computation. In this section, we explore partitioning AlexNet at each layer between the mobile and cloud. In this section, we use Wi-Fi as the wireless network configuration.

Each bar in Figure 6a represents the end-to-end latency of AlexNet, partitioned after each layer. Similarly, each bar in Figure 6b represents the mobile energy consumption of Alexnet, partitioned after each layer. Partitioning computation after a specific layer means executing the DNN on the mobile up to that layer, transferring the output of that layer to the cloud via wireless network, and executing the remaining layers in the cloud. The leftmost bar represents sending the original input for cloud-only processing. As partition point moves from left to right, more layers are executed on the mobile device thus there is an increasingly larger mobile processing component. The rightmost bar is the latency of executing the entire DNN locally on the mobile device.

**Partition for Latency –** If partitioning at the front-end, the data transfer dominates the end-to-end latency, which is consistent with our observation in Section 4.2 that the data size is the largest at the early stage of the DNN. Partitioning at the back-end provides better performance since the application can minimize the data transfer overhead, while taking ad-

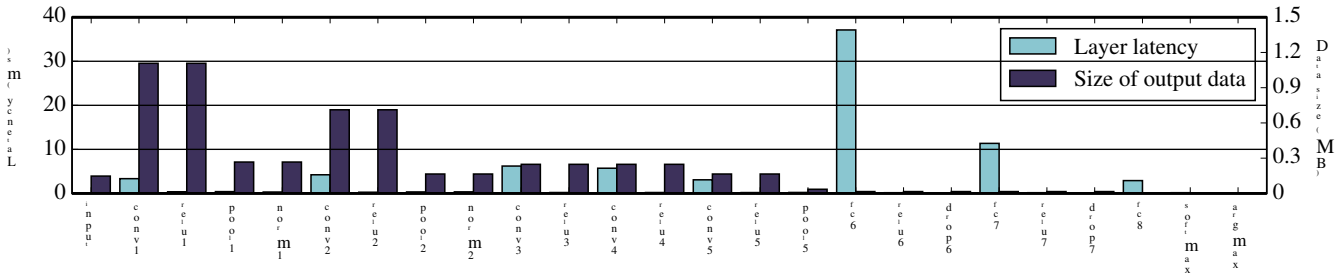
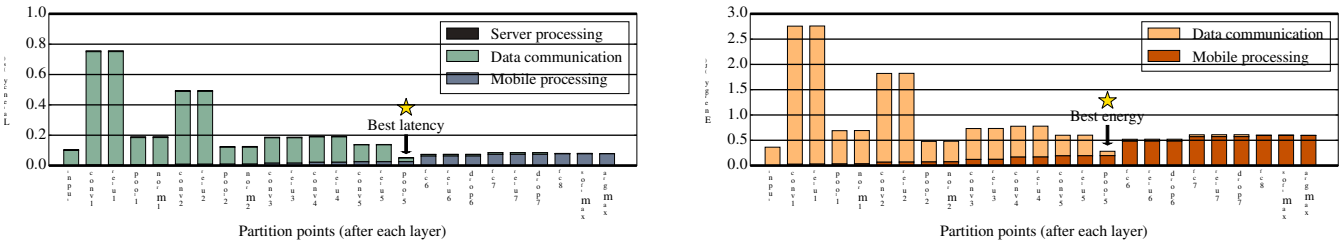



图 5：每层执行后的每层执行时间（浅色左侧条）和数据大小（深色右侧条）（输入 为了AlexNet 中的下一层）。数据大小急剧增加然后减少，而计算量通常通过网络的执行而增加。




**(a) AlexNet latency** **(b) AlexNet energy consumption**

图 6：选择不同分区点时的端到端延迟和移动能耗。每一层执行后都被认为是一个分区点。如果 DNN 在 X 轴上标记的层之后进行分区，则每个条形代表总延迟 (a) 或移动能量 (b)。最左边的条代表仅限云处理，最右边的条代表仅限移动处理。注释了最佳延迟和移动能量的划分点。

的情况，增加了计算量。另一方面，全连接层比网络中的卷积层慢一个数量级。最耗时的层是层 `fc6`，它是 DNN 深处的全连接层，占用总执行时间的 45%。

数据大小变化——图 5 中的右侧条（深色）显示了每层输出数据的大小，这也是下一层的输入。前三个卷积层（`conv1`、`conv2` 和 `conv3`）在输入特征图上应用数百个过滤器以提取有趣的特征时，会生成大量输出数据（显示为最大的黑条）。数据大小在激活层 (`relu1` - `relu5`) 中保持恒定。池化层通过取最大值来总结相邻特征的区域，从而将数据大小急剧减少高达 4.7×。网络较深处的全连接层 (`fc6` - `fc8`) 逐渐减少数据大小，直到最后的 softmax 层 (`softmax`) 和 argmax 层 (`argmax`) 将数据减少为一个分类标签。

***Key Observations*** – 1) 根据其类型和在网络中的位置，每一层都有不同的计算和数据配置文件。 2）移动GPU上的卷积层和池化层的延迟相对较小，而全连接层的延迟较高。 3）卷积层和池化层大多位于网络的前端，而全连接层则位于后端。 4）随着卷积层增加数据，然后池化层减少数据，前端层逐渐减少数据大小。最后几层的数据大小小于原始输入。 5）数据大小普遍减少的结果

前端的每层移动延迟通常较高，这表明在移动和云之间的 DNN 中间进行计算分区的独特机会。

### 4.3 层粒度计算划分

4.2 节中的分析表明，神经网络中存在一些有趣的点来划分计算。在本节中，我们将探讨在移动设备和云之间的每一层对 AlexNet 进行分区。在本节中，我们使用 Wi-Fi 作为无线网络配置。

图 6a 中的每个条形代表 AlexNet 的端到端延迟，在每一层之后进行分区。类似地，图 6b 中的每个条形代表 Alexnet 的移动能耗，在每一层之后进行划分。在特定层之后进行分区计算意味着在移动设备上执行 DNN 直到该层，通过无线网络将该层的输出传输到云端，并在云端执行其余层。最左边的条代表发送原始输入以进行仅云处理。随着分区点从左向右移动，在移动设备上执行更多层，因此移动处理组件越来越大。最右边的条是在移动设备上本地执行整个 DNN 的延迟。

延迟分区——如果在前端分区，则数据传输主导端到端延迟，这与我们在 4.2 节中观察到的数据大小在 DNN 早期阶段最大的情况一致。后端分区提供了更好的性能，因为应用程序可以最大限度地减少数据传输开销，同时采取ad-

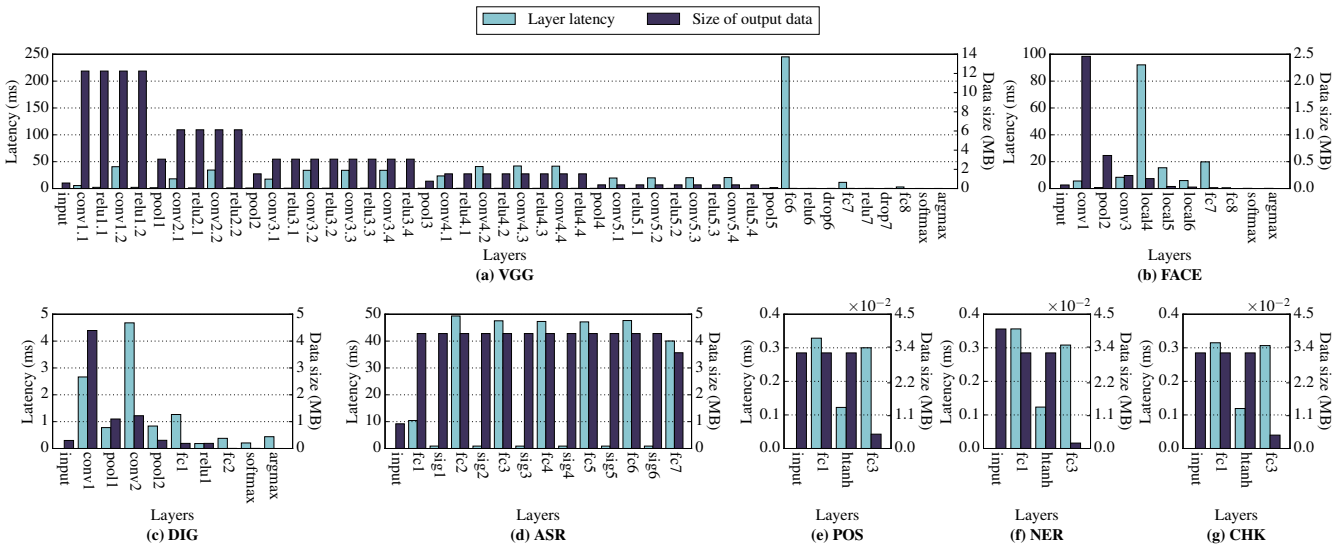



**Figure 7: The per layer latency on the mobile GPU (left light-color bar) and size of data (right dark-color bar) after each layer's execution.**

vantage of the powerful server to execute the more compute-heavy layers at the back-end. In the case of AlexNet using the mobile GPU and Wi-Fi, partitioning between the last pooling layer (`pool5`) and the first fully-connected layer (`fc6`) achieves the lowest latency, as marked in Figure 6a, improving 2.0× over cloud-only processing.

**Partition for Energy –** Similar to latency, due to the high energy cost of wireless data transfer, transferring the input for cloud-only processing is not the most energy-efficiency approach. As marked in Figure 6b, partitioning in the middle of the DNN achieves the best mobile energy consumption, 18% more energy efficient than the cloud-only approach.

***Key Observations* –** Partitioning at the layer granularity can provide significant latency and energy efficiency improvements. For AlexNet using the GPU and Wi-Fi, the best partition points are between the intermediate layers of the DNN.

### 4.4 Generalizing to More DNNs

We expand our investigation to 7 more intelligent applications to study their data and computation characteristics and their impact on computation partitioning opportunity. We use the DNNs provided in the Tonic suite [9], as well as VGG, a state-of-the-art image classification DNN, and LTE as the wireless network configuration. Details about the benchmarks are listed in Table 3. We count the number of layers of each DNN starting from the first non-input layer to the last layer, including `argmax` if present.

**CV Applications –** The three remaining computer vision DNNs (VGG, FACE and DIG) have similar characteristics as AlexNet (Figure 5), as shown in Figures 7a –7c. The front-end layers are convolution layers increasing data, and pooling layers reducing data. The data size in the back-end layers are similar or smaller than the original input data. The latency for the back-end layers are higher than most of the

**Table 3: Benchmark Specifications**

| App | Abbr. | Network | Input | Layers |
|---|---|---|---|---|
| Image classification | IMC | AlexNet [21] | Image | 24 |
| | VGG | VGG [26] | Image | 46 |
| Facial recognition | FACE | DeepFace [27] | Image | 10 |
| Digit recognition | DIG | MNIST [28] | Image | 9 |
| Speech recognition | ASR | Kaldi [29] | Speech features | 13 |
| Part-of-speech tagging | POS | SENNA [30] | Word vectors | 3 |
| Named entity recognition | NER | SENNA [30] | Word vectors | 3 |
| Word chunking | CHK | SENNA [30] | Word vectors | 3 |

front-end layers (e.g., `fc6` is the most time-consuming layer in VGG), except for DIG where convolution layers are most time-consuming. Similar to AlexNet, these characteristics indicate partitioning opportunities in the middle of the DNN. Figure 8a shows that the partition point for best latency for VGG is in the intermediate layers. In addition, Figures 8a - 8c show that different CV applications have different partition points for best latency, and Figures 9a - 9c show the different partition points for best energy for these DNNs.

**ASR and NLP Applications –** The remaining four DNNs in the suite (ASR, POS, NER and CHK) only consist of fully-connected layers and activation layers. The layer breakdowns are shown in Figures 7d - 7g, where, throughout the execution, layers of the same type incur similar latency and the data size stay relatively constant except for the very first and last layer of each DNN. These DNNs do not have data-increasing layers (i.e., convolution layers) or data-reducing layers (i.e., pooling layers). As a result, there only exist opportunities for partitioning the computation at the extremities of these networks. Figures 8d - 8g and Figures 9d - 9g show the different partition points for best latency and energy for these DNNs, respectively. There are data communication components in the right-most bars (mobile-only processing) for these applications because the output of the DNN is sent to the cloud for post-processing steps required by these applications.

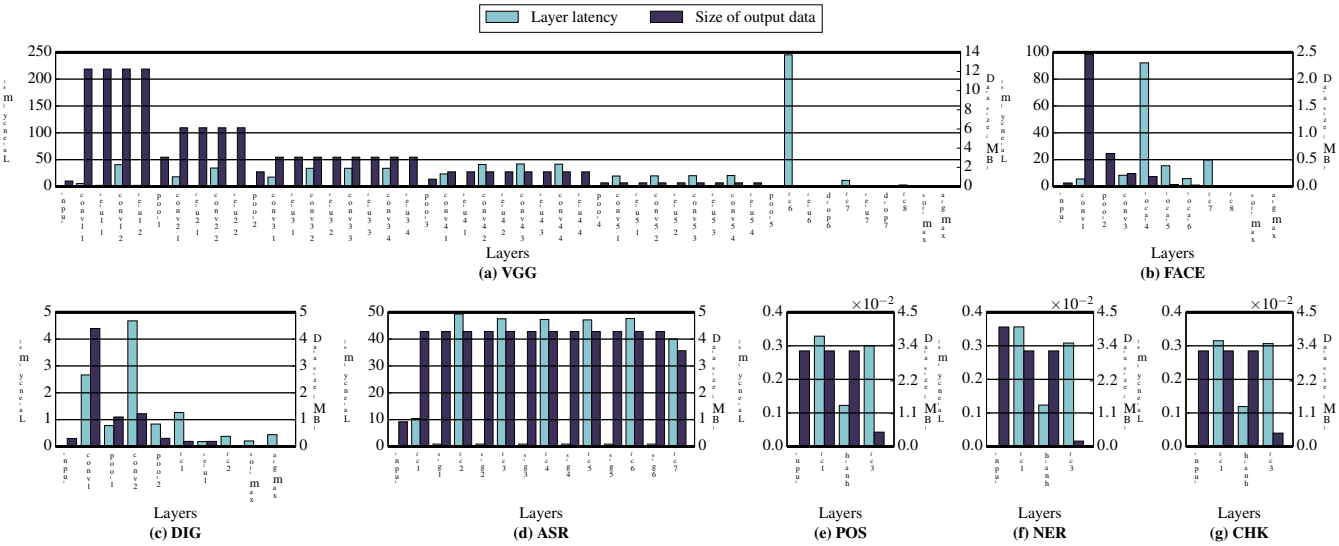



图 7：每层执行后移动 GPU 上的每层延迟（左侧浅色条）和数据大小（右侧深色条）。

利用强大的服务器在后端执行计算量更大的层。在 AlexNet 使用移动 GPU 和 Wi-Fi 的情况下，最后一个池化层（pool5）和第一个全连接层（fc6）之间的分区实现了最低延迟，如图 6a 所示，与纯云处理相比提高了 2.0×。

能源分区——与延迟类似，由于无线数据传输的能源成本很高，传输输入仅用于云处理并不是最节能的方法。如图 6b 所示，在 DNN 中间进行分区可实现最佳的移动能耗，比纯云方法节能 18%。

***Key Observations*** – 层粒度的分区可以显着提高延迟和能源效率。对于使用 GPU 和 Wi-Fi 的 AlexNet，最佳划分点位于 DNN 的中间层之间。

### 4.4 推广到更多的 DNN

我们将调查范围扩大到 7 个更智能的应用程序，以研究它们的数据和计算特性以及它们对计算分区机会的影响。我们使用 Tonic 套件 [9] 中提供的 DNN，以及 VGG（一种最先进的图像分类 DNN）和 LTE 作为无线网络配置。表 3 列出了有关基准的详细信息。我们计算每个 DNN 的层数，从第一个非输入层到最后一层，包括 argmax（如果存在）。

CV 应用——剩下的三个计算机视觉 DNN（VGG、FACE 和 DIG）与 AlexNet（图 5）具有相似的特征，如图 7a –7c 所示。前端层是增加数据的卷积层和减少数据的池化层。后端层的数据大小与原始输入数据相似或较小。后端层的延迟高于大多数层的延迟

表 3：基准规格

| App | Abbr. | Network | Input | Layers |
|---|---|---|---|---|
| Image classification | IMC | AlexNet [21] | Image | 24 |
| | VGG | VGG [26] | Image | 46 |
| Facial recognition | FACE | DeepFace [27] | Image | 10 |
| Digit recognition | DIG | MNIST [28] | Image | 9 |
| Speech recognition | ASR | Kaldi [29] | Speech features | 13 |
| Part-of-speech tagging | POS | SENNA [30] | Word vectors | 3 |
| Named entity recognition | NER | SENNA [30] | Word vectors | 3 |
| Word chunking | CHK | SENNA [30] | Word vectors | 3 |

前端层（例如，fc6 是 VGG 中最耗时的层），但 DIG 除外，其中卷积层最耗时。与 AlexNet 类似，这些特征表明 DNN 中间的分区机会。图 8a 显示 VGG 最佳延迟的划分点位于中间层。此外，图 8a - 8c 显示不同的 CV 应用程序具有不同的最佳延迟划分点，图 9a - 9c 显示了这些 DNN 的最佳能量的不同划分点。

ASR 和 NLP 应用——套件中的其余四个 DNN（ASR、POS、NER 和 CHK）仅由全连接层和激活层组成。层分解如图 7d - 7g 所示，其中在整个执行过程中，相同类型的层会产生类似的延迟，并且除了每个 DNN 的第一层和最后一层之外，数据大小保持相对恒定。这些 DNN 没有数据增加层（即卷积层）或数据减少层（即池化层）。因此，只存在在这些网络的末端划分计算的机会。图 8d - 8g 和图 9d - 9g 分别显示了这些 DNN 的最佳延迟和能量的不同划分点。最右边的栏中有这些应用程序的数据通信组件（仅限移动设备处理），因为 DNN 的输出被发送到云端以进行这些应用程序所需的后处理步骤。

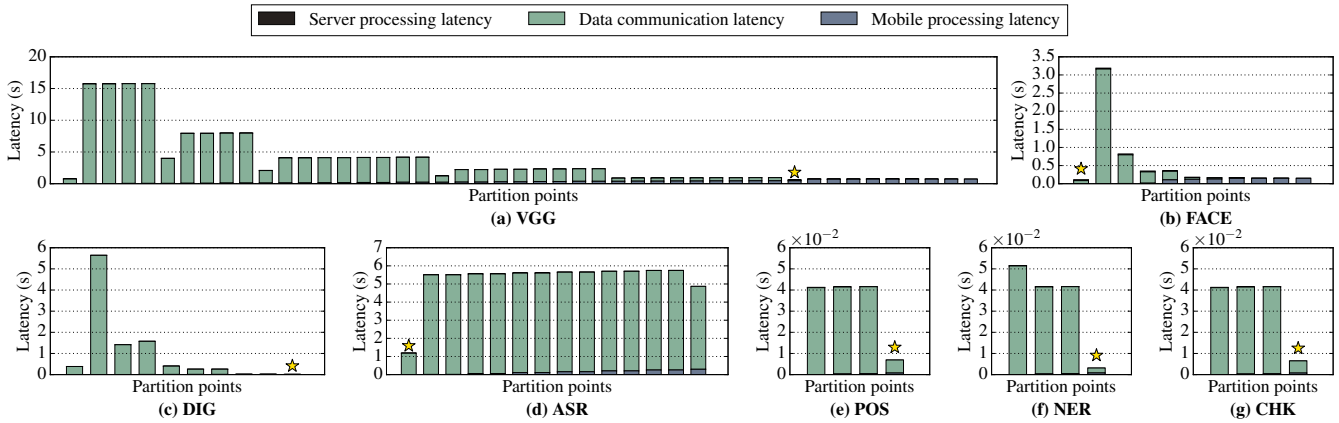



**Figure 8: End-to-end latency when choosing different partition points. Each bar represents the end-to-end latency if the DNN is partitioned after each layer, where the left-most bar represents cloud-only processing (i.e., partitioning at the beginning) while the right-most bar represents mobile-only execution (i.e., partitioning at the end). The wireless network configuration is LTE. The partition points for best latency are each marked by ★.**

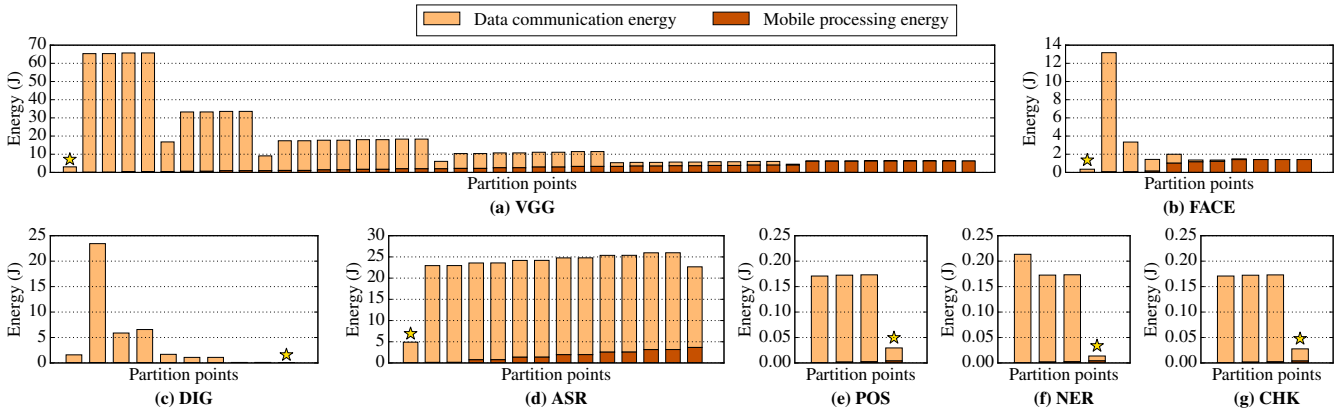



**Figure 9: Mobile energy consumption when choosing different partition points. Each bar represents the mobile energy consumption if the DNN is partitioned after each layer, where the left-most bar represents cloud-only processing (i.e., partitioning at the beginning) while the right-most bar represents mobile-only execution (i.e., partitioning at the end). The wireless network configuration is LTE. The partition points for best energy are each marked by ★.**

***Key Observations*** – 1) In DNNs with convolution and pooling layers (e.g. Computer Vision applications), the data size increases after convolution layers and decreases after pooling layers, while the per-layer computation generally increases through the execution. 2) DNNs with only fully-connected layers of similar size and activation layers see small variations in per-layer latency and data size (e.g., ASR and NLP DNNs). 3) The best way to partition a DNN depends on its topology and constituent layers. Computer vision DNNs sometimes have better partition points in the middle of the DNN, while it is more beneficial to partition at the beginning or the end for ASR and NLP DNNs. The strong variations in the best partition point suggest there is a need for a system to partition DNN computation between the mobile and cloud based on the neural network architecture.

## 5. Neurosurgeon

The best partition point for a DNN architecture depends on the DNN's topology, which manifests itself in the computation and data size variations of each layer. In addition, dynamic factors such as state of the wireless network and datacenter load affect the best partition point even for the same DNN architecture. For example, mobile devices' wireless connections often experience high variances [31], directly affecting the data transfer latency. Datacenters typically experience diurnal load patterns [32], leading to high variance in its DNN query service time. Due to these dynamic factors, there is a need for an automatic system to intelligently select the best point to partition the DNN to optimize for end-to-end latency or mobile device energy consumption. To address this need, we present the design of `Neurosurgeon`, an intelligent DNN partitioning engine. `Neurosurgeon` consists of a deployment phase and a runtime system that manages the partitioned execution of an intelligent application. Figure 10 shows the design of `Neurosurgeon`, which has two stages: deployment and runtime.

**At Deployment –** `Neurosurgeon` profiles the mobile device and the server to generate performance prediction models for the spectrum of DNN layer types (enumerated in Section 4.1). Note that `Neurosurgeon`'s profiling is application

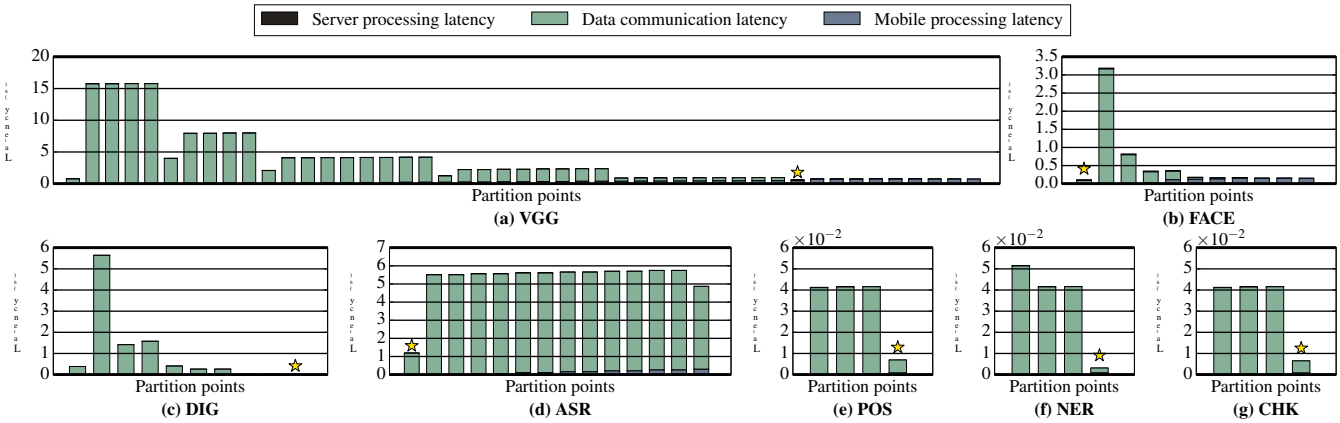



图 8：选择不同分区点时的端到端延迟。如果 DNN 在每一层之后进行分区，则每个条形代表端到端延迟，其中最左边的条形代表仅云处理（即开始时的分区），而最右边的条形代表仅移动执行（即最后的分区）。无线网络配置为 LTE。最佳延迟的划分点均由 ★ 标记。

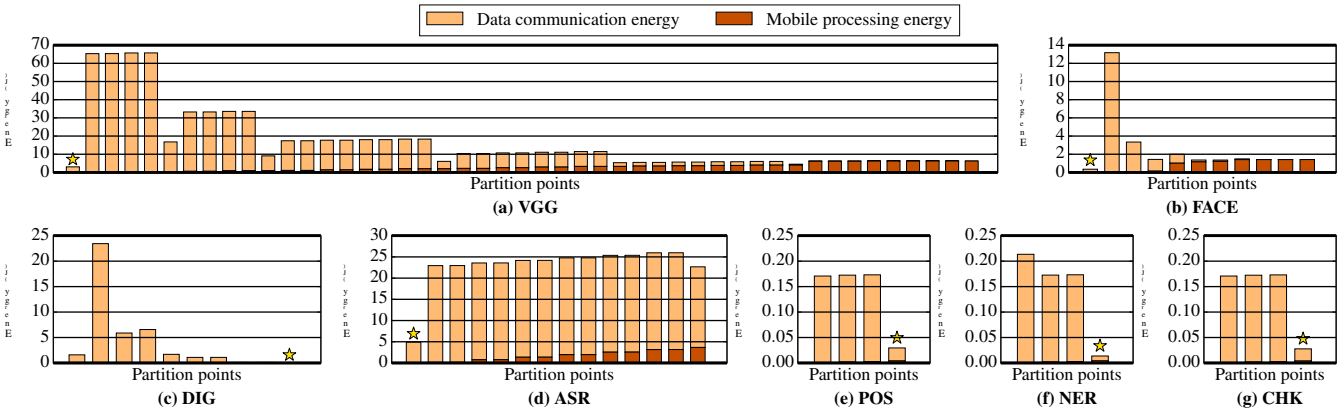



图9：选择不同分区点时的移动能耗。如果 DNN 在每一层之后进行分区，则每个条形代表移动能耗，其中最左边的条形代表仅云处理（即开始时的分区），而最右侧的条形代表仅移动执行（即最后的分区）。无线网络配置为 LTE。最佳能量的划分点分别用 ★ 标记。

***Key Observations*** – 1）在具有卷积层和池化层的DNN中（例如计算机视觉应用），数据大小在卷积层之后增加，在池化层之后减少，而每层计算量通常在执行过程中增加。 2）仅具有相似大小和激活层的全连接层的 DNN 在每层延迟和数据大小方面存在微小变化（例如，ASR 和 NLP DNN）。 3）划分 DNN 的最佳方法取决于其拓扑和组成层。计算机视觉 DNN 有时在 DNN 中间有更好的划分点，而对于 ASR 和 NLP DNN 在开头或结尾进行划分更为有利。最佳划分点的巨大变化表明，需要一个系统根据神经网络架构在移动端和云端之间划分 DNN 计算。

## 5. 神经外科医生

DNN 架构的最佳划分点取决于 DNN 的拓扑，这体现在每层的计算和数据大小变化上。此外，dynamic

即使对于相同的 DNN 架构，无线网络状态和数据中心负载等自然因素也会影响最佳划分点。例如，移动设备的无线连接经常会遇到很大的差异[31]，直接影响数据传输延迟。数据中心通常会经历昼夜负载模式[32]，导致其 DNN 查询服务时间存在很大差异。由于这些动态因素，需要一个自动系统来智能地选择划分 DNN 的最佳点，以优化端到端延迟或移动设备能耗。为了满足这一需求，我们提出了智能 DNN 分区引擎 `Neurosurgeon` 的设计。 `Neurosurgeon`由部署阶段和管理智能应用程序的分区执行的运行时系统组成。图 10 显示了 `Neurosurgeon` 的设计，它有两个阶段：部署和运行时。

在部署时 - `Neurosurgeon` 分析移动设备和服务器，以生成 DNN 层类型范围的性能预测模型（第 4.1 节中列举）。请注意，`Neurosurgeon` 的分析是应用程序

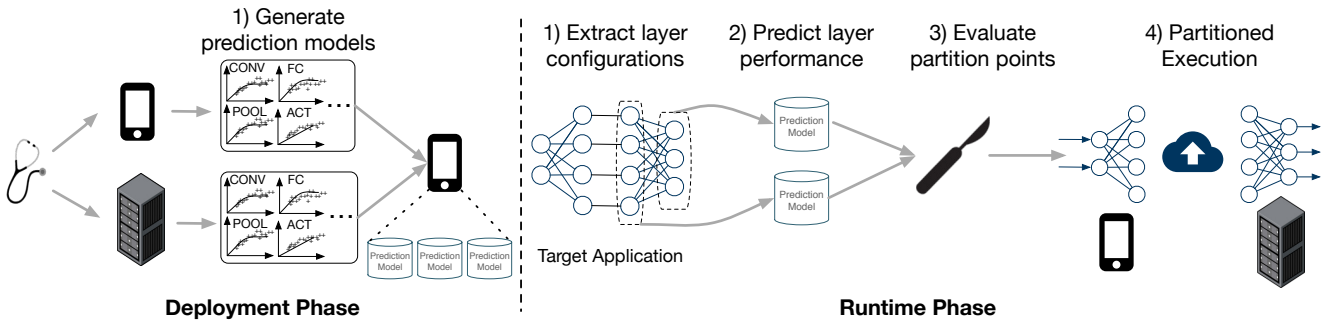



**Figure 10: Overview of Neurosurgeon. At deployment, Neurosurgeon generates prediction models for each layer type. During runtime, Neurosurgeon predicts each layer's latency/energy cost based on the layer's type and configuration, and selects the best partition point based on various dynamic factors.**

agnostic and only needs to be done once for a given set of mobile and server platforms; per-application profiling is not needed. This set of prediction models are stored on the mobile device and later used to predict the latency and energy cost of each layer (Section 5.1).

**During Runtime –** During the execution of an DNN-based intelligent application on the mobile device, Neurosurgeon dynamically decides the best partition point for the DNN. As illustrated in Figure 10, the steps are as follows: 1) Neurosurgeon analyzes and extracts the DNN architecture's layer types and configurations; 2) the system uses the stored layer performance prediction models to estimate the latency and energy consumption for executing each layer on the mobile and cloud; 3) with these predictions, combined with the current wireless connection bandwidth and datacenter load level, Neurosurgeon selects the best partition point, optimizing for best end-to-end latency or best mobile energy consumption; 4) Neurosurgeon executes the DNN, partitioning work between the mobile and cloud.

### 5.1 Performance Prediction Model

Neurosurgeon models the per-layer latency and the energy consumption of arbitrary neural network architecture. This approach allows Neurosurgeon to estimate the latency and energy consumption of a DNN's constituent layers without executing the DNN.

We observe that for each layer type, there is a large latency variation across layer configurations. Thus, to construct the prediction model for each layer type, we vary the configurable parameters of the layer and measure the latency and power consumption for each configuration. Using these profiles, we establish a regression model for each layer type to predict the latency and power of the layer based on its configuration. We describe each layer's regression model variables later in this section. We use GFLOPS (Giga Floating Point Operations per Second) as our performance metric. Based on the layer type, we use either a logarithmic or linear function as the regression function. The logarithmic-based regression is used to model the performance plateau as the computation requirement of the layer approaches the limit of the available hardware resources.

**Convolution, local and pooling** layers' configurable parameters include the input feature map dimension, number, size and stride of the filters. The regression model for convolution layer is based on two variables: the number of features in the input feature maps, and $(filter\ size/stride)^2 \times (\#\ of\ filters)$, which represents the amount of computation applied to each pixel in the input feature maps. For local and pooling layers, we use the size of the input and output feature maps as the regression model variables.

In a **fully-connected** layer, the input data is multiplied by the learned weight matrix to generate the output vector. We use the number of input neurons and number of output neurons as the regression model variables. **Softmax** and **argmax** layers are handled similarly.

**Activation** layers have fewer configurable parameters compared to other layers because activation layers have a one-to-one mapping between their input data and output. We use the number of neurons as the regression model variable. We apply the same approach to **normalization** layers.

As previously mentioned, it is a one-time profiling step required for each mobile and server hardware platform to generate a set of prediction models. The models enable Neurosurgeon to estimate the latency and energy cost of each layer based its configuration, which allows Neurosurgeon to support future neural network architectures without additional profiling overhead.

### 5.2 Dynamic DNN Partitioning

Utilizing the layer performance prediction models, Neurosurgeon dynamically selects the best DNN partition points, as described in Algorithm 1. The algorithm has two-steps: analysis of the target DNN and partition point selection.

**Analysis of the Target DNN –** Neurosurgeon analyzes the target DNN's constituent layers, and uses the prediction models to estimate, for each layer, the latency on mobile and cloud, and power consumption on the mobile. Specifically, at lines 11 and 12 of Algorithm 1, Neurosurgeon extracts each layer's type and configuration ($L_i$) and uses the regression models to predict the latency of executing layer $L_i$ on

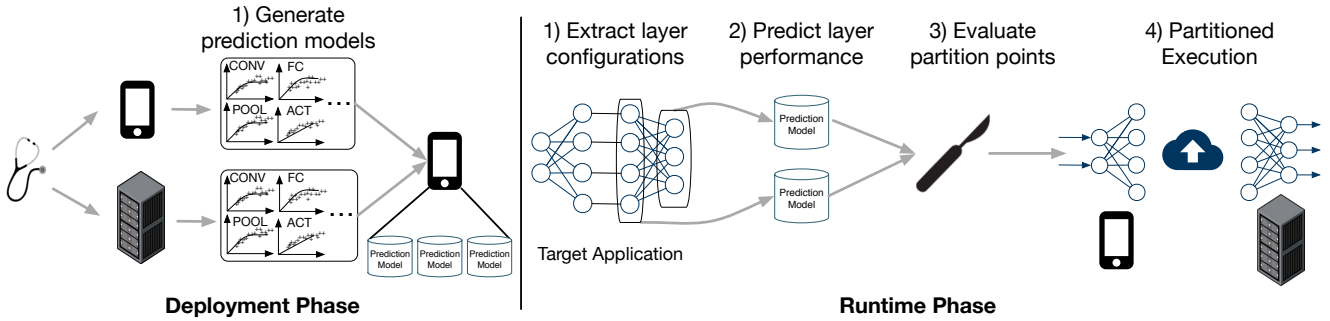



图 10：Neurosurgeon 概述。在部署时，Neurosurgeon 为每个层类型生成预测模型。在运行时，Neuro- surgeon 根据层的类型和配置预测每层的延迟/能量成本，并根据各种动态因素选择最佳划分点。

不可知论，对于一组给定的移动和服务器平台只需执行一次；不需要每个应用程序分析。这组预测模型存储在移动设备上，随后用于预测每层的延迟和能源成本（第 5.1 节）。

运行时期间 – 在移动设备上执行基于 DNN 的智能应用程序期间，Neurosur- geon 动态决定 DNN 的最佳划分点。如图10所示，步骤如下：1）Neurosurgeon分析并提取DNN架构的层类型和配置； 2) 系统使用存储的层性能预测模型来估计在移动设备和云上执行每一层的延迟和能耗； 3）根据这些预测，结合当前的无线连接带宽和数据中心负载水平，Neurosurgeon选择最佳划分点，优化最佳端到端延迟或最佳移动能耗； 4) Neurosurgeon 执行 DNN，在移动设备和云端之间划分工作。

### 5.1 性能预测模型

Neurosurgeon 对任意神经网络架构的每层延迟和能耗进行建模。这种方法允许 Neurosurgeon 在不执行 DNN 的情况下估计 DNN 组成层的延迟和能耗。

我们观察到，对于每种层类型，层配置之间存在很大的延迟变化。因此，为了构建每个层类型的预测模型，我们改变层的可配置参数并测量每个配置的延迟和功耗。使用这些配置文件，我们为每个层类型建立一个回归模型，以根据其配置来预测该层的延迟和功率。我们将在本节后面描述每一层的回归模型变量。我们使用 GFLOPS（每秒千兆浮点运算）作为我们的性能指标。根据图层类型，我们使用对数函数或线性函数作为回归函数。当层的计算要求接近可用硬件资源的限制时，基于对数的回归用于对性能平台进行建模。

卷积层、局部层和池化层的可配置参数包括输入特征图维度、滤波器的数量、大小和步长。卷积层的回归模型基于两个变量：输入特征图中的特征数量，以及$(filter\ size/stride)^2 \times (\#\ of\ filters)$，它表示应用于输入特征图中每个像素的计算量。对于局部层和池化层，我们使用输入和输出特征图的大小作为回归模型变量。

在全连接层中，输入数据乘以学习的权重矩阵以生成输出向量。我们使用输入神经元的数量和输出神经元的数量作为回归模型变量。 Softmax 和 argmax 层的处理方式类似。

与其他层相比，激活层的可配置参数较少，因为激活层的输入数据和输出之间具有一对一的映射。我们使用神经元的数量作为回归模型变量。我们对标准化层应用相同的方法。

如前所述，每个移动和服务器硬件平台都需要一次性分析步骤来生成一组预测模型。这些模型使 Neurosurgeon 能够根据其配置来估计每层的延迟和能量成本，这使得 Neurosur- geon 能够支持未来的神经网络架构，而无需额外的分析开销。

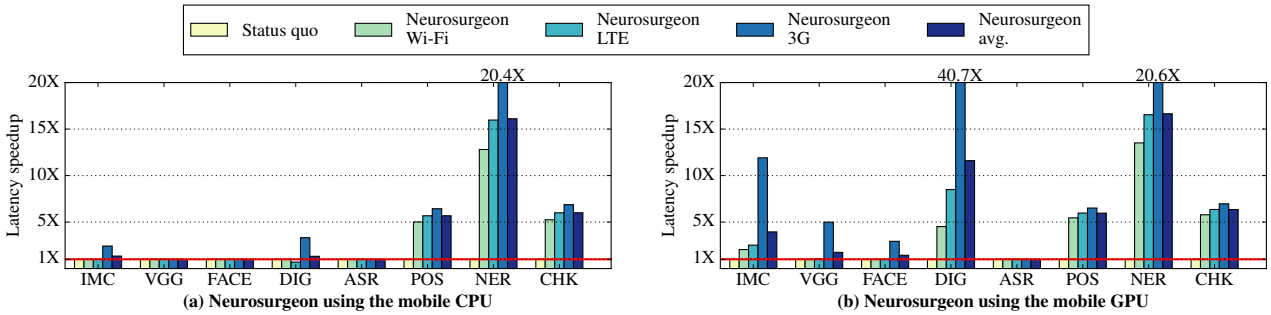
### 5.2 动态DNN分区

利用层性能预测模型，Neuro- surgeon 动态选择最佳 DNN 划分点，如算法 1 中所述。该算法有两个步骤：目标 DNN 分析和划分点选择。

目标 DNN 分析 – Neurosurgeon 分析目标 DNN 的组成层，并使用预测模型来估计每一层的移动和云延迟以及移动设备的功耗。具体来说，在算法 1 的第 11 行和第 12 行，Neurosurgeon 提取每个层的类型和配置 $(L_i)$，并使用回归模型来预测执行层 $L_i$ 的延迟。

**Table 4: Neurosurgeon's partition point selections for best end-to-end latency. Green block indicates Neurosurgeon makes the optimal partition choice and white block means a suboptimal partition point is picked. On average, Neurosurgeon achieves within 98.5% of the optimal performance.**

| Mobile | Wireless network | Benchmarks | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | IMC | VGG | FACE | DIG | ASR | POS | NER | CHK |
| CPU | Wi-Fi | input | input | input | input | input | fc3 | | |
| | LTE | input | input | input | argmax | input | fc3 | | |
| | 3G | argmax | input | input | argmax | input | fc3 | | |
| GPU | Wi-Fi | pool5 | input | input | argmax | input | fc3 | | |
| | LTE | argmax | argmax | input | argmax | input | fc3 | | |
| | 3G | argmax | argmax | argmax | argmax | input | fc3 | | |




(a) Neurosurgeon using the mobile CPU

(b) Neurosurgeon using the mobile GPU

**Figure 11: Latency speedup achieved by Neurosurgeon normalized to status quo approach (executing entire DNN in the cloud). Results for three wireless networks (Wi-Fi, LTE and 3G) and mobile CPU and GPU are shown here. Neurosurgeon improves the end-to-end DNN inference latency by 3.1× on average (geometric mean) and up to 40.7×.**

mobile ($TM_i$) and cloud ($TC_i$), while taking into consideration of current datacenter load level ($K$). Line 13 estimates the power of executing layer $L_i$ on the mobile device ($PM_i$) and line 14 calculates the wireless data transfer latency ($TU_i$) based on the latest wireless network bandwidth.

**Partition Point Selection –** Neurosurgeon then selects the best partition point. The candidate points are after each layer. Lines 16 and 18 evaluate the performance when partitioning at each candidate point and select the point for either best end-to-end latency or best mobile energy consumption. Because of the simplicity of the regression models, this evaluation is lightweight and efficient.

**Algorithm 1** Neurosurgeon DNN partitioning algorithm

1: **Input:**
2: $N$: number of layers in the DNN
3: $\{L_i | i = 1 \cdots N\}$: layers in the DNN
4: $\{D_i | i = 1 \cdots N\}$: data size at each layer
5: $f, g(L_i)$: regression models predicting the latency and power of executing $L_i$
6: $K$: current datacenter load level
7: $B$: current wireless network uplink bandwidth
8: $PU$: wireless network uplink power consumption
9: **procedure** PARTITIONDECISION
10: **for each** $i$ *in* $1 \cdots N$ **do**
11: $TM_i \leftarrow f_{mobile}(L_i)$
12: $TC_i \leftarrow f_{cloud}(L_i, K)$
13: $PM_i \leftarrow g_{mobile}(L_i)$
14: $TU_i \leftarrow D_i / B$
15: **if** $OptTarget == latency$ **then**
16: **return** $\arg\min_{j=1\cdots N}(\sum_{i=1}^{j} TM_i + \sum_{k=j+1}^{N} TC_k + TU_j)$
17: **else if** $OptTarget == energy$ **then**
18: **return** $\arg\min_{j=1\cdots N}(\sum_{i=1}^{j} TM_i \times PM_i + TU_j \times PU)$

### 5.3 Partitioned Execution

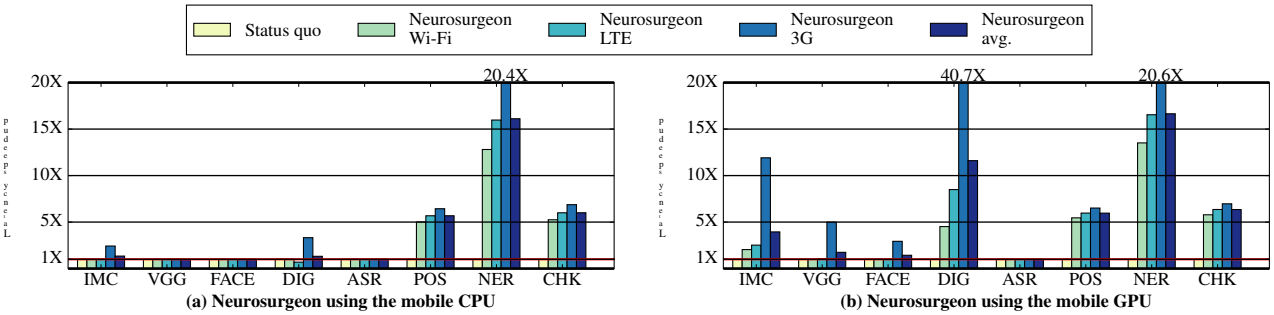
We prototype Neurosurgeon by creating modified instances of Caffe [18] to serve as our mobile-side (NSmobile) and server-side (NSserver) infrastructures. Through these two variations of Caffe, we implement our client-server interface using Thrift [33], an open source flexible RPC interface for inter-process communication. To allow for flexibility in the dynamic selection of partition points, both NSmobile and NSserver host complete DNN models, and partition points are enforced by NSmobile and NSserver runtime. Given a partition decision by NSmobile, execution begins on the mobile device and cascades through the layers of the DNN leading up to that partition point. Upon completion of that layer, NSmobile sends the output of that layer from the mobile device to NSserver residing on the server side. NSserver then executes the remaining DNN layers. Upon the completion of the DNN execution, the final result is sent back to NSmobile on the mobile device from NSserver. Note that there is exactly one partition point within the DNN for which information is sent from the mobile device to the cloud.

## 6. Evaluation

We evaluate Neurosurgeon using 8 DNNs (Table 3) as our benchmarks across Wi-Fi, LTE and 3G wireless connections with both CPU-only and GPU mobile platforms. We demonstrate Neurosurgeon achieves significant end-to-end latency and mobile energy improvements over the status quo cloud-only approach (Sections 6.1 and 6.2). We then compare Neurosurgeon against MAUI [34], a well-known

表 4：Neurosurgeon 的分区点选择以获得最佳端到端延迟。绿色块表示 Neurosurgeon 做出了最佳划分选择，白色块表示选择了次优划分点。平均而言，Neurosurgeon 达到最佳性能的 98.5% 以内。

| Mobile | Wireless network | Benchmarks | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | IMC | VGG | FACE | DIG | ASR | POS | NER | CHK |
| CPU | Wi-Fi | input | input | input | input | input | fc3 | | |
| | LTE | input | input | input | argmax | input | fc3 | | |
| | 3G | argmax | input | input | argmax | input | fc3 | | |
| GPU | Wi-Fi | pool5 | input | input | argmax | input | fc3 | | |
| | LTE | argmax | argmax | input | argmax | input | fc3 | | |
| | 3G | argmax | argmax | argmax | argmax | input | fc3 | | |




(a) Neurosurgeon using the mobile CPU　　(b) Neurosurgeon using the mobile GPU

图 11：通过归一化至现状方法（在云中执行整个 DNN）的 Neurosurgeon 实现延迟加速。此处显示了三种无线网络（Wi-Fi、LTE 和 3G）以及移动 CPU 和 GPU 的结果。Neurosurgeon 将端到端 DNN 推理延迟平均提高了 3.1×（几何平均值），最高可达 40.7×。

移动 ($TM_i$) 和云 ($TC_i$)，同时考虑当前数据中心负载水平 ($K$)。第 13 行估计移动设备上执行层 $L_i$ 的功率 ($PM_i$)，第 14 行根据最新的无线网络带宽计算无线数据传输延迟 ($TU_i$)。

分区点选择 – Neurosurgeon 然后选择最佳分区点。候选点位于每一层之后。第 16 行和第 18 行评估在每个候选点进行分区时的性能，并选择最佳端到端延迟或最佳移动能耗的点。由于回归模型的简单性，这种评估是轻量级且高效的。

**Algorithm 1** Neurosurgeon DNN partitioning algorithm

1: **Input:**
2: $N$: number of layers in the DNN
3: $\{L_i | i = 1 \cdots N\}$: layers in the DNN
4: $\{D_i | i = 1 \cdots N\}$: data size at each layer
5: $f, g(L_i)$: regression models predicting the latency and power of executing $L_i$
6: $K$: current datacenter load level
7: $B$: current wireless network uplink bandwidth
8: $PU$: wireless network uplink power consumption
9: **procedure** PARTITIONDECISION
10: **for each** $i$ *in* $1 \cdots N$ **do**
11: $TM_i \leftarrow f_{mobile}(L_i)$
12: $TC_i \leftarrow f_{cloud}(L_i, K)$
13: $PM_i \leftarrow g_{mobile}(L_i)$
14: $TU_i \leftarrow D_i / B$
15: **if** $OptTarget == latency$ **then**
16: **return** $\arg\min_{j=1\cdots N}(\sum_{i=1}^{j} TM_i + \sum_{k=j+1}^{N} TC_k + TU_j)$
17: **else if** $OptTarget == energy$ **then**
18: **return** $\arg\min_{j=1\cdots N}(\sum_{i=1}^{j} TM_i \times PM_i + TU_j \times PU)$

### 5.3 分区执行

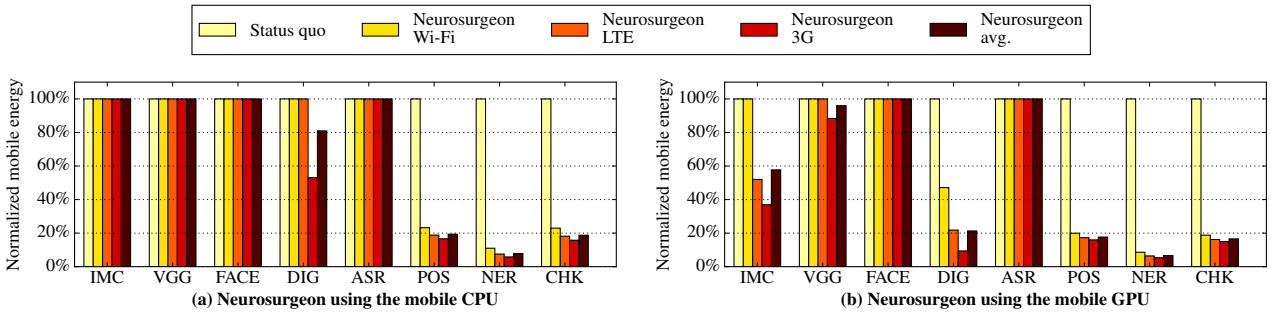
我们通过创建 Caffe [18] 的修改实例来原型化 Neurosurgeon，以用作我们的移动端 (NSmobile) 和服务器端 (NSserver) 基础设施。通过 Caffe 的这两个变体，我们使用 Thrift [33] 实现客户端-服务器接口，Thrift 是一个用于进程间通信的开源灵活 RPC 接口。为了实现动态选择分区点的灵活性，NSmo-、bile和NSserver都托管完整的DNN模型，并且分区点由NSmobile和NSserver运行时强制执行。给定 NSmobile 的分区决策，执行开始在移动设备上，并级联通过 DNN 的各层，直至该分区点。该层完成后，NSmobile 将该层的输出从移动设备发送到驻留在服务器端的 NSserver。NSserver 然后执行剩余的 DNN 层。 DNN 执行完成后，最终结果从 NSserver 发送回移动设备上的 NSmobile。请注意，DNN 中只有一个分区点，信息从移动设备发送到云端。

## 六、评价

我们使用 8 个 DNN（表 3）作为 Wi-Fi、LTE 和 3G 无线连接与仅 CPU 和 GPU 移动平台的基准来评估 Neurosurgeon。我们证明，与现状的纯云方法（第 6.1 和 6.2 节）相比，Neurosurgeon 实现了显着的端到端延迟和移动能源改进。然后我们将 Neurosurgeon 与 MAUI [34] 进行比较，MAUI [34] 是一个众所周知的

**Table 5: `Neurosurgeon` partition point selections for best mobile energy consumption. Green block indicates `Neurosurgeon` makes the optimal partition choice and white block means a suboptimal partition point is picked. On average, `Neurosurgeon` achieves a mobile energy reduction within 98.8% of the optimal reduction.**

| Mobile | Wireless network | Benchmarks | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | IMC | VGG | FACE | DIG | ASR | POS | NER | CHK |
| CPU | Wi-Fi | input | input | input | input | input | fc3 | | |
| | LTE | input | input | input | input | input | fc3 | | |
| | 3G | input | input | input | argmax | input | fc3 | | |
| GPU | Wi-Fi | input | input | input | argmax | input | fc3 | | |
| | LTE | pool5 | input | input | argmax | input | fc3 | | |
| | 3G | argmax | argmax | input | argmax | input | fc3 | | |




(a) Neurosurgeon using the mobile CPU

(b) Neurosurgeon using the mobile GPU

**Figure 12: Mobile energy consumption achieved by `Neurosurgeon` normalized to status quo approach (executing entire DNN in the cloud). Results for three wireless networks (Wi-Fi, LTE and 3G) and mobile CPU and GPU are shown here. `Neurosurgeon` reduces the mobile energy consumption by 59.5% on average (geometric mean) and up to 94.7%.**

computation offloading framework (Section 6.3). We also evaluate `Neurosurgeon`'s robustness to variations in wireless network connections (Section 6.4) and server load (Section 6.5), demonstrating the need for such a dynamic runtime system. Finally, we evaluate the datacenter throughput improvement `Neurosurgeon` achieves by pushing compute out of the cloud to the mobile device (Section 6.6).

### 6.1 Latency Improvement

**Partition Point Selection –** Table 4 summarizes the partition points selected by `Neurosurgeon` optimizing for latency across the 48 configurations (i.e., 8 benchmarks, 3 wireless network types, mobile CPU and GPU). The green cells indicate when `Neurosurgeon` selects the optimal partition point and achieves the best speedup while the white cells indicate `Neurosurgeon` selects a suboptimal point. `Neurosurgeon` selects the best partition point for 44 out of the 48 configurations. The mispredictions occur because the partition points and its associated performance are very close to one another and thus a small difference in `Neurosurgeon`'s latency prediction shifts the selection. Across all benchmarks and configurations, `Neurosurgeon` achieves latency speedup within 98.5% of optimal speedup.

**Latency Improvement –** Figure 11 shows `Neurosurgeon`'s latency improvement over the status quo approach, across the 8 benchmarks on Wi-Fi, LTE, and 3G. Figure 11a shows the latency improvement when applying `Neurosurgeon` to a mobile platform equipped with a CPU, and Figure 11b shows that of a mobile platform with a GPU. For CV applications, `Neurosurgeon` identifies the best partition points for 20 out of 24 cases and achieves significant latency speedups, especially when the mobile GPU is available. For the NLP applications, `Neurosurgeon` achieves significant latency speedups even when Wi-Fi is available. For ASR, `Neurosurgeon` successfully identifies that it is best to execute the DNN entirely on the server and, therefore `Neurosurgeon` performs similar to the status quo for that particular benchmark. Across all benchmarks and configurations, `Neurosurgeon` achieves a latency speedup of 3.1× on average and up to 40.7× over the status quo approach.

### 6.2 Energy Improvement

**Partition Point Selection –** Table 5 summarizes the partition points identified by `Neurosurgeon` for best mobile energy. `Neurosurgeon` selects the best partition point for 44 out of the 48 configurations. For the suboptimal choices, `Neurosurgeon` consumes 24.2% less energy on average than the status quo approach.

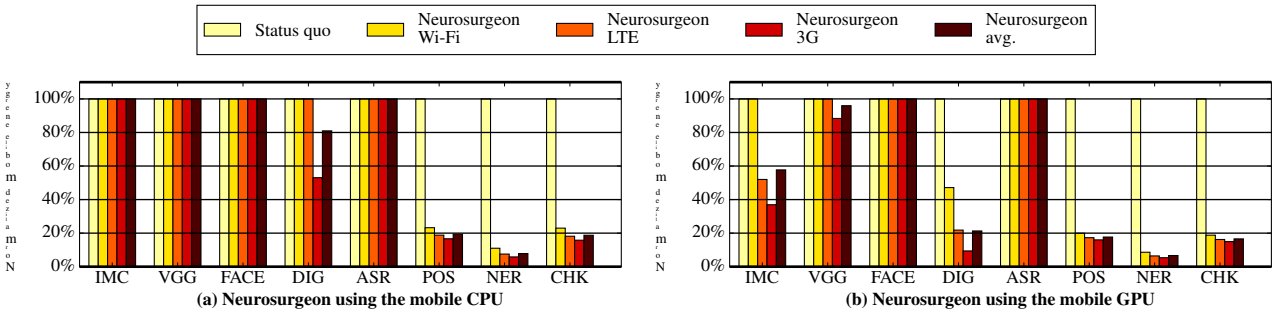
**Energy Improvement –** Figure 12 shows the mobile energy consumption achieved by `Neurosurgeon`, normalized to the status quo approach. Figure 12a and 12b present results for CPU-only mobile platform and GPU-equipped mobile platform, respectively. When optimizing for best energy consumption, `Neurosurgeon` achieves on average a 59.5% reduction in mobile energy and up to 94.7% reduction over the status quo. Similar to the improvement for latency, the energy reduction is also higher for most benchmarks when the mobile platform is equipped with a GPU.

### 6.3 Comparing Neurosurgeon to MAUI

In this section, we compare `Neurosurgeon` to MAUI [34], a general offloading framework. Note that MAUI is control-

表 5：最佳移动能耗的 Neurosurgeon 划分点选择。绿色块表示 Neurosurgeon 做出了最佳划分选择，白色块表示选择了次优划分点。平均而言，Neurosurgeon 实现的移动能源减少量在最佳减少量的 98.8% 以内。

| Mobile | Wireless network | Benchmarks | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | IMC | VGG | FACE | DIG | ASR | POS | NER | CHK |
| CPU | Wi-Fi | input | input | input | input | input | fc3 | | |
| | LTE | input | input | input | input | input | fc3 | | |
| | 3G | input | input | input | argmax | input | fc3 | | |
| GPU | Wi-Fi | input | input | input | argmax | input | fc3 | | |
| | LTE | pool5 | input | input | argmax | input | fc3 | | |
| | 3G | argmax | argmax | input | argmax | input | fc3 | | |




(a) Neurosurgeon using the mobile CPU

(b) Neurosurgeon using the mobile GPU

图 12：通过 Neurosurgeon 归一化至现状方法（在云中执行整个 DNN）实现的移动能耗。此处显示了三种无线网络（Wi-Fi、LTE 和 3G）以及移动 CPU 和 GPU 的结果。 Neurosurgeon 移动能耗平均降低 59.5%（几何平均值），最高可达 94.7%。

计算泛洪框架（第 6.3 节）。我们还评估了 Neurosurgeon 对无线网络连接（第 6.4 节）和服务器负载（第 6.5 节）变化的鲁棒性，证明了对这种动态运行时系统的需求。最后，我们评估通过将计算从云端推送到移动设备来实现的数据中心吞吐量改进 Neurosurgeon（第 6.6 节）。

### 6.1 延迟改善

分区点选择 - 表 4 总结了 Neurosurgeon 在 48 种配置（即 8 个基准、3 种无线网络类型、移动 CPU 和 GPU）中优化延迟时选择的分区点。绿色单元格表示 Neurosurgeon 选择最佳分割点并实现最佳加速，而白色单元格表示 Neurosurgeon 选择次优点。Neurosurgeon 为 48 个配置中的 44 个选择最佳划分点。发生错误预测是因为分区点及其相关性能彼此非常接近，因此 Neuro- surgeon 的延迟预测中的微小差异会改变选择。在所有基准测试和配置中，Neurosurgeon 在最佳加速的 98.5% 内实现了延迟加速。

延迟改进 - 图 11 显示了在 Wi-Fi、LTE 和 3G 的 8 个基准测试中，Neurosurgeon 相对于现状方法的延迟改进。图 11a 显示了将 Neurosurgeon 应用于配备 CPU 的移动平台时的延迟改进，图 11b 显示了配备 GPU 的移动平台的延迟改进。对于 CV 应用，Neurosurgeon 识别 24 个案例中的 20 个的最佳划分点，并实现显着的延迟加速，特别是当移动 GPU 可用时。对于 NLP 应用程序，即使 Wi-Fi 可用，Neurosurgeon 也能实现显着的延迟加速。对于 ASR，Neurosurgeon 成功地确定最好完全在服务器上执行 DNN，因此 Neu- rosurgeon 的性能与该特定基准的现状类似。在所有基准测试和配置中，Neurosurgeon 的延迟加速平均为 3.1×，与现状方法相比最高可达 40.7×。

### 6.2 能源改进

分配点选择——表5总结了由Neurosurgeon确定的最佳移动能量的分配点。 Neurosurgeon 为 48 个配置中的 44 个选择最佳划分点。对于次优选择，Neurosurgeon 比现状方法平均消耗 24.2% 的能量。

能源改进——图 12 显示了 Neurosurgeon 实现的移动能源消耗，标准化为现状方法。图 12a 和 12b 分别显示了仅使用 CPU 的移动平台和配备 GPU 的移动平台的结果。在优化最佳能源消耗时，Neurosurgeon 的移动能源平均减少 59.5%，比现状减少高达 94.7%。与延迟的改进类似，当移动平台配备 GPU 时，大多数基准测试的能耗降低也更高。

### 6.3 神经外科医生与 MAUI 的比较

在本节中，我们将 Neurosurgeon 与通用卸载框架 MAUI [34] 进行比较。请注意，MAUI 是控制-

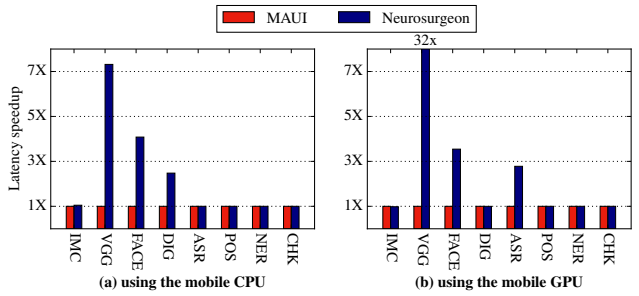



**Figure 13: Latency speedup achieved by `Neurosurgeon` vs. MAUI [34]. For MAUI, we assume the optimal programmer annotation that achieves minimal program state transfer. `Neurosurgeon` outperforms MAUI by up to 32× and 1.9× on average.**

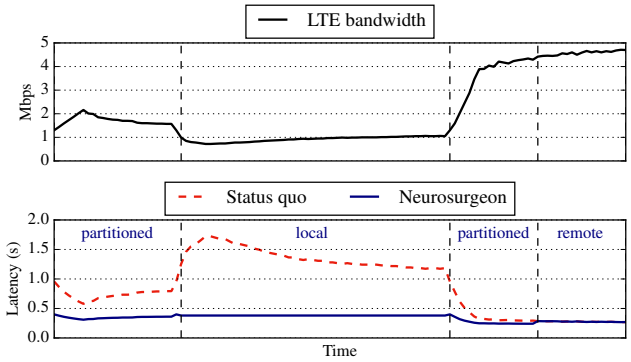



**Figure 14: The top graph shows bandwidth variance using a LTE network. The bottom graph shows the latency of AlexNet (IMC) of the status quo and `Neurosurgeon`. `Neurosurgeon`'s decisions are annotated on the bottom graph. `Neurosurgeon` provides consistent latency by adjusting its partitioned execution based on the available bandwidth.**

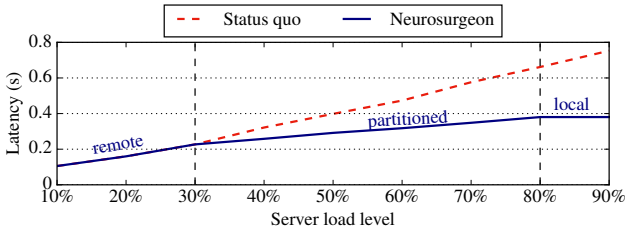



**Figure 15: `Neurosurgeon` adjusts its partitioned execution as the result of varying datacenter load.**

centric, reasoning and making decisions about regions of code (functions), whereas `Neurosurgeon` is data-centric, making partition decisions based on the structure of the data topology that can differ even if the same code region (function) is called.

Figure 13 presents the latency speedup achieved by `Neurosurgeon` normalized to MAUI when executing the 8 DNN benchmarks, averaged across three wireless network types. Figure 13a presents the result when applying MAUI and `Neurosurgeon` on a CPU-only mobile platform and Figure 13b presents the result on a mobile platform equipped with a GPU. In this experiment, we assume that for MAUI, programmers have optimally annotated the minimal program states that need to be transferred.

Figure 13 shows that `Neurosurgeon` significantly outperforms MAUI on the computer vision applications. For the NLP applications, both `Neurosurgeon` and MAUI correctly decide that local computation on the mobile device is optimal. However, MAUI makes incorrect offloading choices for more complicated scenarios (e.g., VGG, FACE, DIG and ASR). This is because MAUI relies on past invocation of a certain DNN layer type to predict the latency and data size of the future invocations of that layer type, leading to mispredictions. This control-centric prediction mechanism is not suitable for DNN layers because the latency and data size of layers of the same type can be drastically different within one DNN, and `Neurosurgeon`'s DNN analysis step and prediction model correctly captures this variation. For instance, in VGG, the input data size for the first and second convolution layers are significantly different: 0.57MB for `conv1.1`, and 12.25MB for `conv1.2`. For the mobile CPU and LTE, MAUI decides to offload the DNN before `conv1.2` due to its misprediction, uploading large amount of data and resulting in a 20.5× slowdown over the status quo approach. Meanwhile, `Neurosurgeon` successfully identifies that for this case it is best to execute the DNN entirely in the cloud, and thus achieves similar performance as the status quo and a 20.5× speedup over MAUI.

### 6.4 Network Variation

In this section, we evaluate `Neurosurgeon`'s resilience to real-world measured wireless network variations. In Figure 14, the top graph shows measured wireless bandwidth of T-Mobile LTE network over a period of time. The bottom graph shows the end-to-end latency of the status quo approach and `Neurosurgeon` executing AlexNet (IMC) on the mobile CPU platform. Annotated on the bottom graph is `Neurosurgeon`'s dynamic execution choice, categorized as either local, remote or partitioned. The status quo approach is highly susceptible to network variations and consequently the application suffers significant latency increases during the low bandwidth phase. Conversely, `Neurosurgeon` successfully mitigates the effects of large variations and provides consistent low latency by shifting partition choice to adjust the amount of data transfer based on the available bandwidth.

### 6.5 Server Load Variation

In this section, we evaluate how `Neurosurgeon` makes dynamic decision as the server load varies. Datacenters typically experience diurnal load patterns and high server utilization leads to increased service time for DNN queries. `Neurosurgeon` determines the best partition point based on the current server load level obtained by periodically pinging

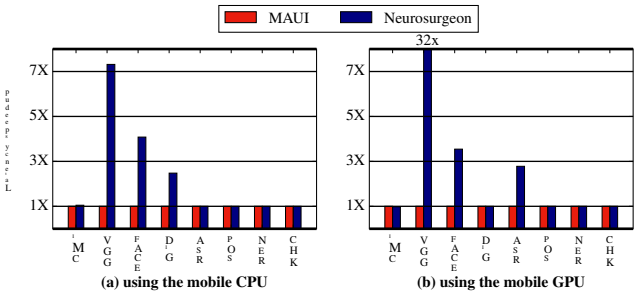



图 13：Neurosurgeon 与 MAUI [34] 实现的延迟加速。对于 MAUI，我们假设实现最小程序状态传输的最佳程序员注释。 Neurosurgeon 的平均性能比 MAUI 高出 32× 和 1.9×。

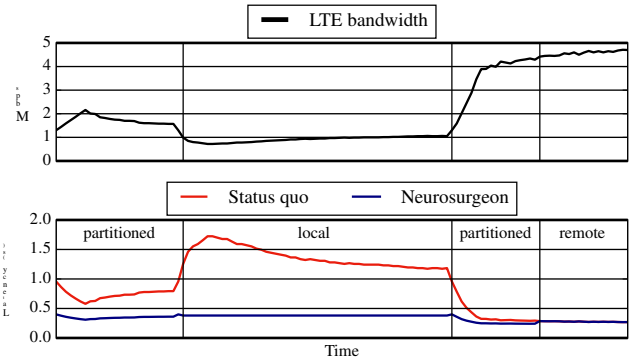



图 14：上图显示了使用 LTE 网络的带宽变化。下图显示了 AlexNet (IMC) 现状和 Neurosurgeon 的延迟。 Neurosurgeon 的决策在底部图表上进行了注释。 Neurosurgeon 通过根据可用带宽调整其分区执行来提供一致的延迟。

以数据为中心，对代码区域（函数）进行推理和决策，而 Neurosurgeon 以数据为中心，根据数据拓扑结构做出分区决策，即使调用相同的代码区域（函数），数据拓扑结构也可能有所不同。

图 13 显示了执行 8 个 DNN 基准测试时，通过归一化为 MAUI 的 Neu- rosurgeon 实现的延迟加速，这是三种无线网络类型的平均值。图 13a 显示了在仅使用 CPU 的移动平台上应用 MAUI 和 Neurosurgeon 时的结果，图 13b 显示了在配备 GPU 的移动平台上的结果。在这个实验中，我们假设对于 MAUI，程序员已经最佳地注释了需要传输的最小程序状态。

图 13 显示 Neurosurgeon 在计算机视觉应用上的表现明显优于 MAUI。对于 NLP 应用程序，Neurosurgeon 和 MAUI 都正确地确定移动设备上的本地计算是最佳的。然而，MAUI 对于更复杂的场景（例如 VGG、FACE、DIG 和 ASR）会做出错误的卸载选择。这是因为 MAUI 依赖于过去对某个 DNN 层类型的调用来预测该层类型未来调用的延迟和数据大小，从而导致错误预测。这种以控制为中心的预测机制并不适合 DNN 层，因为同一类型的层的延迟和数据大小在一个 DNN 内可能截然不同，而 Neurosurgeon 的 DNN 分析步骤和预测模型正确地捕获了这种变化。例如，在 VGG 中，第一和第二卷积层的输入数据大小显着不同：conv1.1 为 0.57MB，conv1.2 为 12.25MB。对于移动 CPU和LTE，MAUI决定在conv1.2之前卸载DNN，因为它的预测错误，上传大量数据并导致比现状方法慢20.5×。同时，Neurosurgeon 成功地确定，对于这种情况，最好完全在云端执行 DNN，从而实现与现状类似的性能，并且比 MAUI 加速 20.5×。

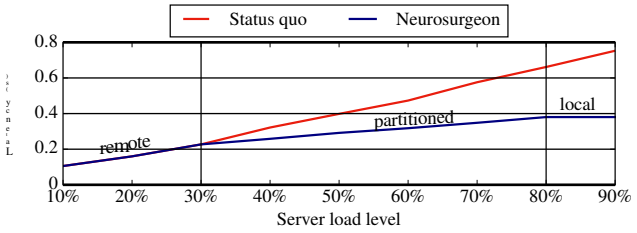



图 15：Neurosurgeon 根据数据中心负载变化调整其分区执行。

## 6.4 网络变化

在本节中，我们评估 Neurosurgeon 对现实世界测量的无线网络变化的恢复能力。在图 14 中，顶部图表显示了一段时间内测量的 T-Mobile LTE 网络无线带宽。下图显示了现状方法和在移动 CPU 平台上执行 AlexNet (IMC) 的 Neurosurgeon 的端到端延迟。底部图表中注释的是 Neurosurgeon 的动态执行选择，分为本地、远程或分区。现状方法非常容易受到网络变化的影响，因此应用程序在低带宽阶段会遭受显着的延迟增加。相反，Neurosur- geon 成功地减轻了大变化的影响，并通过改变分区选择以根据可用带宽调整数据传输量来提供一致的低延迟。

## 6.5 服务器负载变化

在本节中，我们评估 Neurosurgeon 如何在服务器负载变化时做出动态决策。数据中心通常会经历昼夜负载模式，而高服务器利用率会导致 DNN 查询的服务时间增加。 Neu- rosurgeon 根据定期 ping 得到的当前服务器负载水平确定最佳分区点

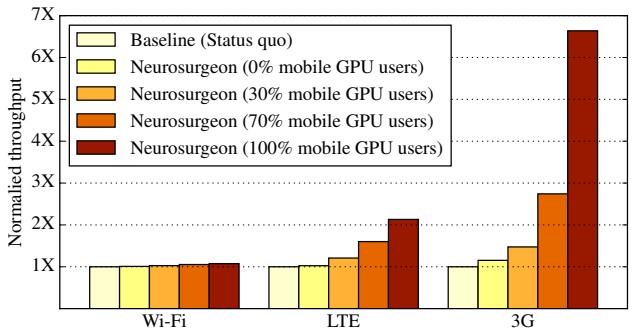



**Figure 16: Datacenter throughput improvement achieved by `Neurosurgeon` over the status quo approach. Higher throughput improvement is achieved by `Neurosurgeon` for cellular networks (LTE and 3G) and as more mobile devices are equipped with GPUs.**

the server during idle period, and thus avoids long latency caused by high user demand and the resulting high load.

Figure 15 presents the end-to-end latency of AlexNet (IMC) achieved by the status quo approach and `Neurosurgeon` as the server load increases. The mobile device is equipped with a CPU and transfers data via Wi-Fi. As shown in the figure, the status quo approach does not dynamically adapt to varying server load and thus suffers from significant performance degradation when the server load is high. The end-to-end latency of the status quo approach increases from 105ms to 753ms as the server approaches its peak load level. On the other hand, by taking server load into consideration, `Neurosurgeon` dynamically adapts the partition point. In Figure 15, two vertical dashed lines represent the points where `Neurosurgeon` changes its selection: from complete cloud execution at low load, to partitioning the DNN between mobile and cloud at medium load, and eventually completely onloading to mobile at peak load. Regardless of the server load, `Neurosurgeon` keeps the end-to-end latency of executing image classification below 380ms. By considering server load and its impact on the server performance, `Neurosurgeon` consistently delivers the best latency regardless of the variation in server load.

### 6.6 Datacenter Throughput Improvement

`Neurosurgeon` onloads part or all of the computation from the cloud to mobile devices to improve end-to-end latency and reduce mobile energy consumption. This new compute paradigm reduces the computation required on the datacenter, leading to shorter query service time and higher query throughput. In this section, we evaluate `Neurosurgeon`'s effectiveness in this aspect. We use BigHouse [38] to compare the achieved datacenter throughput between status quo and `Neurosurgeon`. The incoming DNN queries are composed evenly of the 8 DNNs in the benchmark suite. We use the measured mean service time of DNN queries combined with Google web search query distribution for the query inter-arrival rate.

Figure 16 presents the datacenter throughput improvement achieved by `Neurosurgeon`, normalized to the baseline status quo approach of executing the entire computation on the server. Each cluster presents results for a given wireless network type. Within each cluster, the first bar represents the status quo cloud-only approach, while the other four bars represent `Neurosurgeon` with different compositions of the mobile hardware. For example, "30% Mobile GPU users" indicates 30% of the incoming requests are from mobile devices equipped with a GPU while the remaining 70% are from devices equipped only with a CPU.

When the mobile clients are connected to the server via fast Wi-Fi network, `Neurosurgeon` achieves on average 1.04× throughput improvement. As the wireless connection changes to LTE and 3G, the throughput improvement becomes more significant: 1.43× for LTE and 2.36× for 3G. `Neurosurgeon` adapts its partition choice and pushes larger portions of the DNN computation to the mobile devices as the wireless connection quality becomes less ideal. Therefore the average request query service time is reduced and a higher throughput is achieved in the datacenter. We also observe that as the percentage of mobile devices with GPU increases, `Neurosurgeon` increases the computation onloading from the cloud to mobile, leading to higher datacenter throughput improvement.

## 7. Related Work

Previous research efforts focus on offloading computation from the mobile to cloud. In Table 6, we compare `Neurosurgeon` with the most relevant techniques on properties including whether there is heavy data transfer overhead, data-centric or control-centric partitioning, low run-time overhead, whether application-specific profiling is required, and whether programmer's annotation is needed.

In addition to these key differences, computation partition frameworks have to make predictions as to when to offload computation and the correctness of the prediction dictates the final performance improvements for the application. COMET [35] offloads a thread when its execution time exceeds a pre-defined threshold, ignoring any other information (amount of data to transfer, wireless network available, etc.). Odessa [36] makes computation partition decisions only considering the execution time and data requirements of part of the function, without taking the entire application into consideration. CloneCloud [37] makes the same offloading decisions for all invocations of the same function. MAUI's [34] offloading decision mechanism is better in that it makes predictions for each function invocation separately and considers the entire application when choosing which function to offload. However, MAUI is not applicable for the computation partition performed by `Neurosurgeon` for a number of reasons: 1) MAUI requires a profiling step for each individual application, whereas predictions are required to perform DNN partitioning. `Neurosurgeon` makes deci-

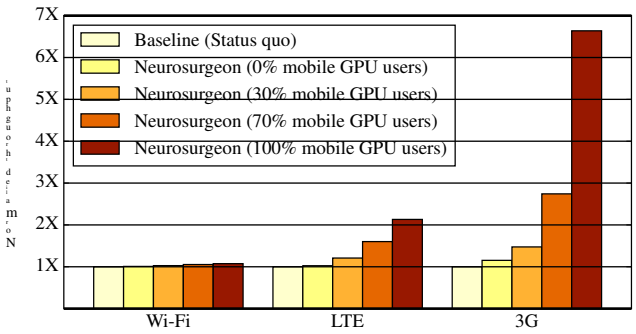



图 16：Neuro- surgeon 相对于现状方法实现的数据中心吞吐量改进。随着越来越多的移动设备配备 GPU，蜂窝网络（LTE 和 3G）的 Neurosurgeon 可以实现更高的吞吐量改进。

服务器在空闲期间，从而避免因用户需求高而导致的长时间延迟和由此产生的高负载。

图 15 显示了随着服务器负载增加，通过现状方法和 Neuro- surgeon 实现的 AlexNet (IMC) 的端到端延迟。移动设备配备CPU并通过Wi-Fi传输数据。如图所示，现状方法不能动态适应不断变化的服务器负载，因此当服务器负载较高时，性能会显着下降。随着服务器接近其峰值负载水平，现状方法的端到端延迟从 105 毫秒增加到 753 毫秒。另一方面，通过考虑服务器负载，Neurosurgeon动态调整分区点。在图 15 中，两条垂直虚线代表 Neurosurgeon 更改其选择的点：从低负载时的完全云执行，到中等负载时在移动设备和云之间划分 DNN，以及最终在峰值负载时完全加载到移动设备。无论服务器负载如何，Neurosurgeon 都能将执行图像分类的端到端延迟保持在 380 毫秒以下。通过考虑服务器负载及其对服务器性能的影响，无论服务器负载如何变化，Neurosurgeon都能始终提供最佳延迟。

### 6.6 数据中心吞吐量提升

Neurosurgeon 将部分或全部计算从云端加载到移动设备，以改善端到端延迟并降低移动能耗。这种新的计算范式减少了数据中心所需的计算，从而缩短了查询服务时间并提高了查询吞吐量。在本节中，我们评估 Neurosurgeon在这方面的有效性。我们使用 BigHouse [38] 来比较现状和 Neurosurgeon 之间实现的数据中心吞吐量。传入的 DNN 查询均匀地由基准套件中的 8 个 DNN 组成。我们使用测得的 DNN 查询平均服务时间与 Google 网络搜索查询分布相结合来计算查询间隔到达率。

图 16 显示了 Neurosurgeon 实现的数据中心吞吐量改进，标准化为在服务器上执行整个计算的基线现状方法。每个集群呈现给定无线网络类型的结果。在每个集群中，第一个条代表现状的纯云方法，而其他四个条代表具有不同移动硬件组成的Neurosurgeon。例如，“30% 移动 GPU 用户”表示 30% 的传入请求来自配备 GPU 的移动设备，而其余 70% 来自仅配备 CPU 的设备。

当移动客户端通过快速 Wi-Fi 网络连接到服务器时，Neurosurgeon 的吞吐量平均提高了 1.04×。随着无线连接变为 LTE 和 3G，吞吐量改进变得更加显着：LTE 为 1.43×，3G 为 2.36×。当无线连接质量变得不太理想时，Neurosurgeon 会调整其分区选择，并将 DNN 计算的较大部分推送到移动设备。因此，减少了平均请求查询服务时间，并在数据中心实现了更高的吞吐量。我们还观察到，随着配备 GPU 的移动设备比例的增加，Neurosurgeon 增加了从云到移动设备的计算负载，从而提高了数据中心吞吐量。

## 七、相关工作

之前的研究工作主要集中在将计算从移动设备转移到云端。在表 6 中，我们将 Neuro- surgeon 与最相关的属性技术进行了比较，包括是否存在大量数据传输开销、以数据为中心或以控制为中心的分区、低运行时开销、是否需要特定于应用程序的分析以及是否需要程序员的注释。

除了这些关键差异之外，计算分区框架还必须预测何时卸载计算，并且预测的正确性决定了应用程序的最终性能改进。当执行时间超过预先定义的阈值时，COMET [35] 会卸载线程，忽略任何其他信息（要传输的数据量、可用的无线网络等）。 Odessa [36]仅考虑部分函数的执行时间和数据要求来做出计算分区决策，而不考虑整个应用程序。 CloneCloud [37] 对同一函数的所有调用做出相同的卸载决策。 MAUI 的 [34] 卸载决策机制更好，因为它单独对每个函数调用进行预测，并在选择卸载哪个函数时考虑整个应用程序。然而，MAUI 不适用于 Neurosurgeon 执行的计算分区，原因如下：1) MAUI 需要对每个单独的应用程序进行分析步骤，而执行 DNN 分区则需要预测。 Neurosurgeon 做出决定-

**Table 6: Comparing Neurosurgeon to popular computation offloading/partition frameworks**

| | MAUI [34] | Comet [35] | Odessa [36] | CloneCloud [37] | Neurosurgeon |
|---|---|---|---|---|---|
| **No need to transfer program state** | | | ✓ | | ✓ |
| **Data-centric compute partitioning** | | | | | ✓ |
| **Low/no runtime overhead** | ✓ | | ✓ | ✓ | ✓ |
| **Requires no application-specific profiling** | | ✓ | | | ✓ |
| **No programmer annotation needed** | | ✓ | ✓ | ✓ | ✓ |
| **Server load sensitive** | | | ✓ | | ✓ |

sions based on the DNN topology without any runtime profiling. 2) MAUI is control-centric, making decisions about regions of code (functions), whereas Neurosurgeon makes partition decisions based on the structure of the data topology that can differ even if the same code region (function) is executed. Layers of a given type (even if mapped to the same function) within one DNN can have significantly different compute and data characteristics. 3) Neurosurgeon transfers only the data that is being processed in contrast to transferring all program state. 4) MAUI requires the programmer to annotate their programs to identify which methods are "offload-able".

In addition to prior work investigating the utilization and efficiency of datacenter systems [39–52], there has been growing interest in building large scale datacenter systems for Deep Neural Network workloads. Various accelerators, such as GPUs, ASICs, and FPGAs, have been proposed for datacenters to better handle DNN computation [9, 53–55]. There has also been effort in designing compact DNNs suitable for the mobile edge. Microsoft and Google explore small-scale DNNs for speech recognition on mobile platforms [56, 57]. MCDNN [58] proposes generating alternative DNN models to trade-off accuracy for performance/energy and choosing to execute either in the cloud or on the mobile. This work investigates intelligent collaboration between the mobile device and cloud for executing traditionally cloud-only large-scale DNNs for reduced latency and energy consumption without sacrificing the DNNs' high prediction accuracy.

## 8. Conclusion

As an essential component of today's intelligent applications, Deep Neural Networks have been traditionally executed in the cloud. In this work, we examine the efficacy of this status quo approach of cloud-only processing and show that it is not always optimal to transfer the input data to the server and remotely execute the DNN. We investigate the compute and data characteristics of 8 DNN architectures spanning computer vision, speech, and natural language processing applications and show the trade-off of partitioning computation at different points within the neural network. With these insights, we develop Neurosurgeon, a system that can automatically partition DNN between the mobile device and cloud at the granularity of neural network layers. Neurosurgeon adapts to various DNN architectures, hardware platforms, wireless connections, and server load levels, and chooses the partition point for best latency and best mobile energy consumption. Across 8 benchmarks, when compared to cloud-only processing, Neurosurgeon achieves on average 3.1× and up to 40.7× latency speedup, reduces mobile energy consumption by on average 59.5% and up to 94.7%, and improves datacenter throughput by on average 1.5× and up to 6.7×.

## 9. Acknowledgment

We thank our anonymous reviewers for their feedback and suggestions. This work was sponsored by ARM, Intel, and the National Science Foundation under grants IIS-VEC-1539011, CCF-SHF-1302682, CNS-CSR-1321047 and NSF CAREER SHF-1553485.

## References

[1] Wearables market to be worth $25 billion by 2019. http://www.ccsinsight.com/press/company-news/2332-wearables-market-to-be-worth-25-billion-by-2019-reveals-ccs-insight. Accessed: 2017-01.

[2] Rapid Expansion Projected for Smart Home Devices, IHS Markit Says. http://news.ihsmarkit.com/press-release/technology/rapid-expansion-projected-smart-home-devices-ihs-markit-says. Accessed: 2017-01.

[3] Intelligent Virtual Assistant Market Worth $3.07Bn By 2020. https://globenewswire.com/news-release/2015/12/17/796353/0/en/Intelligent-Virtual-Assistant-Market-Worth-3-07Bn-By-2020.html. Accessed: 2016-08.

[4] Intelligent Virtual Assistant Market Analysis And Segment Forecasts 2015 To 2022. https://www.hexaresearch.com/research-report/intelligent-virtual-assistant-industry/. Accessed: 2016-08.

[5] Growing Focus on Strengthening Customer Relations Spurs Adoption of Intelligent Virtual Assistant Technology. http://www.transparencymarketresearch.com/pressrelease/intelligent-virtual-assistant-industry.html/. Accessed: 2016-08.

[6] Google Brain. https://backchannel.com/google-search-will-be-your-next-brain-5207c26e4523#.x9n2ajota. Accessed: 2017-01.

[7] Microsoft Deep Learning Outperforms Humans in Image Recognition. http://www.forbes.com/sites/michaelthomsen/2015/02/19/microsofts-deep-learning-project-outperforms-humans-in-image-recognition/. Accessed: 2016-08.

表 6：将 Neurosurgeon 与流行的加载/分区计算框架进行比较

| | MAUI [34] | Comet [35] | Odessa [36] | CloneCloud [37] | Neurosurgeon |
|---|---|---|---|---|---|
| **No need to transfer program state** | | | ✓ | | ✓ |
| **Data-centric compute partitioning** | | | | | ✓ |
| **Low/no runtime overhead** | ✓ | | ✓ | ✓ | ✓ |
| **Requires no application-specific profiling** | | ✓ | | | ✓ |
| **No programmer annotation needed** | | ✓ | ✓ | ✓ | ✓ |
| **Server load sensitive** | | | ✓ | | ✓ |

基于 DNN 拓扑的系统，无需任何运行时分析。 2）MAUI 以控制为中心，对代码（函数）区域做出决策，而 Neurosurgeon 根据数据拓扑结构做出分区决策，即使执行相同的代码区域（函数），数据拓扑结构也可能不同。一个 DNN 中给定类型的层（即使映射到相同的函数）可能具有显着不同的计算和数据特征。 3) Neurosurgeon 仅传输正在处理的数据，而不是传输所有程序状态。 4) MAUI 要求程序员对其程序进行注释，以识别哪些方法是“可卸载的”。

除了先前研究数据中心系统的利用率和效率的工作之外[39-52]，人们对为深度神经网络工作负载构建大规模数据中心系统越来越感兴趣。人们已经为数据中心提出了各种加速器，例如 GPU、ASIC 和 FPGA，以更好地处理 DNN 计算 [9, 53–55]。人们还致力于设计适合移动边缘的紧凑型 DNN。 Microsoft 和 Google 探索用于移动平台上语音识别的小型 DNN [56, 57]。 MCDNN [58]建议生成替代的 DNN 模型，以权衡性能/能源的准确性，并选择在云端或移动设备上执行。这项工作研究了移动设备和云之间的智能协作，用于执行传统上仅在云中的大规模 DNN，以减少延迟和能耗，同时又不牺牲 DNN 的高预测精度。

## 八、结论

作为当今智能应用程序的重要组成部分，深度神经网络传统上是在云端执行的。在这项工作中，我们研究了这种仅云处理的现状方法的有效性，并表明将输入数据传输到服务器并远程执行 DNN 并不总是最佳的。我们研究了涵盖计算机视觉、语音和自然语言处理应用的 8 个 DNN 架构的计算和数据特征，并展示了神经网络内不同点的分区计算的权衡。有了这些见解，我们开发了 Neurosur- geon，一个可以在神经网络层的粒度上自动在移动设备和云之间划分 DNN 的系统。 Neurosurgeon适应各种DNN架构、硬件平台、无线连接和服务器负载水平，并选择最佳的划分点张力和最佳移动能耗。在 8 个基准测试中，与纯云处理相比，Neuro- surgeon 实现了平均 3.1× 和高达 40.7× 的延迟加速，平均降低了 59.5% 和高达 94.7% 的移动能耗，并将数据中心吞吐量平均提高了 1.5× 和高达 6.7×。

## 9.致谢

我们感谢匿名审稿人的反馈和建议。这项工作由 ARM、英特尔和国家科学基金会赞助，资助金额为 IIS-VEC-1539011、CCF-SHF-1302682、CNS-CSR-1321047 和 NSF CAREER SHF-1553485。

## 参考

[1] 到 2019 年，可穿戴设备市场价值将达到 250 亿美元。 http://www.ccsinsight.com/press/company-news/2332-wearables-market-to-be-worth-25-billion-by-2019-reveals-ccs-insight。访问时间：2017 年 1 月。 [2] IHS Markit 表示，智能家居设备预计将快速扩张。 http://news.ihsmarkit.com/press-release/technology/rapid-expansion-projected-smart-home-devices-ihs-markit-says。访问时间：2017 年 1 月。 [3] 到 2020 年，智能虚拟助手市场价值将达到 30.7 亿美元。https://globenewswire.com/news-release/2015/12/17/796353/0/en/Intelligent-Virtual-Assistant-Market-Worth-3-07Bn-By-2020.html。访问时间：2016 年 8 月。 [4] 2015年至2022年智能虚拟助理市场分析和细分市场预测。https://www.hexaresearch.com/research-report/intelligent-virtual-assistant-industry/。访问时间：2016 年 8 月。 [5] 对加强客户关系的日益关注促进了智能虚拟助理技术的采用。 http://www.transparencymarketresearch.com/pressrelease/intelligent-virtual-assistant-industry.html/。访问时间：2016 年 8 月。

[6] 谷歌大脑。 https://backchannel.com/google-search-will-be-your-next-brain-5207c26e4523#.x9n2ajota。访问时间：2017 年 1 月。

[7] 微软深度学习在图像识别方面优于人类。 http://www.forbes.com/sites/michaelthomsen/2015/02/19/microsofts-deep-learning-project-outperforms-humans-in-image-recognition/。访问时间：2016 年 8 月。

[8] Baidu Supercomputer. https://gigaom.com/2015/01/14/baidu-has-built-a-supercomputer-for-deep-learning/. Accessed: 2016-08.

[9] Johann Hauswald, Yiping Kang, Michael A. Laurenzano, Quan Chen, Cheng Li, Trevor Mudge, Ronald G. Dreslinski, Jason Mars, and Lingjia Tang. Djinn and tonic: Dnn as a service and its implications for future warehouse scale computers. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, ISCA '15, New York, NY, USA, 2015. ACM.

[10] The 'Google Brain' is a real thing but very few people have seen it. http://www.businessinsider.com/what-is-google-brain-2016-9. Accessed: 2017-01.

[11] Google supercharges machine learning tasks with TPU custom chip. https://cloudplatform.googleblog.com/2016/05/Google-supercharges-machine-learning-tasks-with-custom-chip.html. Accessed: 2017-01.

[12] Apple's Massive New Data Center Set To Host Nuance Tech. http://techcrunch.com/2011/05/09/apple-nuance-data-center-deal/. Accessed: 2016-08.

[13] Apple moves to third-generation Siri back-end, built on open-source Mesos platform. http://9to5mac.com/2015/04/27/siri-backend-mesos/. Accessed: 2016-08.

[14] Matthew Halpern, Yuhao Zhu, and Vijay Janapa Reddi. Mobile cpu's rise to power: Quantifying the impact of generational mobile cpu design trends on performance, energy, and user satisfaction. In *High Performance Computer Architecture (HPCA), 2016 IEEE International Symposium on*, pages 64–76. IEEE, 2016.

[15] Whitepaper: NVIDIA Tegra X1. Technical report. Accessed: 2017-01.

[16] NVIDIA Jetson TK1 Development Kit: Bringing GPU-accelerated computing to Embedded Systems. Technical report. Accessed: 2017-01.

[17] Nvidia's Tegra K1 at the Heart of Google's Nexus 9. http://www.pcmag.com/article2/0,2817,2470740,00.asp. Accessed: 2016-08.

[18] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv preprint arXiv:1408.5093*, 2014.

[19] Qian Wang, Xianyi Zhang, Yunquan Zhang, and Qing Yi. Augem: automatically generate high performance dense linear algebra kernels on x86 cpus. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, page 25. ACM, 2013.

[20] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cuDNN: Efficient Primitives for Deep Learning. *CoRR*, abs/1410.0759, 2014.

[21] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, 2012.

[22] Adam Coates, Brody Huval, Tao Wang, David Wu, Bryan Catanzaro, and Andrew Ng. Deep learning with cots hpc systems. In *Proceedings of the 30th international conference on machine learning*, pages 1337–1345, 2013.

[23] TestMyNet: Internet Speed Test. http://testmy.net/. Accessed: 2015-02.

[24] Watts Up? Power Meter. https://www.wattsupmeters.com/. Accessed: 2015-05.

[25] Junxian Huang, Feng Qian, Alexandre Gerber, Z Morley Mao, Subhabrata Sen, and Oliver Spatscheck. A close examination of performance and power characteristics of 4g lte networks. In *Proceedings of the 10th international conference on Mobile systems, applications, and services*, pages 225–238. ACM, 2012.

[26] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[27] Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato, and Lior Wolf. Deepface: Closing the gap to human-level performance in face verification. In *Computer Vision and Pattern Recognition (CVPR)*, 2014.

[28] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.

[29] Daniel Povey, Arnab Ghoshal, Gilles Boulianne, Lukas Burget, Ondrej Glembek, Nagendra Goel, Mirko Hannemann, Petr Motlicek, Yanmin Qian, Petr Schwarz, et al. The kaldi speech recognition toolkit. In *Proc. ASRU*, 2011.

[30] Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu, and Pavel Kuksa. Natural language processing (almost) from scratch. *The Journal of Machine Learning Research*, 2011.

[31] Ashkan Nikravesh, David R Choffnes, Ethan Katz-Bassett, Z Morley Mao, and Matt Welsh. Mobile network performance from user devices: A longitudinal, multidimensional analysis. In *International Conference on Passive and Active Network Measurement*, pages 12–22. Springer, 2014.

[32] David Lo, Liqun Cheng, Rama Govindaraju, Luiz André Barroso, and Christos Kozyrakis. Towards energy proportionality for large-scale latency-critical workloads. In *ACM SIGARCH Computer Architecture News*, volume 42, pages 301–312. IEEE Press, 2014.

[33] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. Thrift: Scalable cross-language services implementation. *Facebook White Paper*, 5(8), 2007.

[34] Eduardo Cuervo, Aruna Balasubramanian, Dae-ki Cho, Alec Wolman, Stefan Saroiu, Ranveer Chandra, and Paramvir Bahl. Maui: making smartphones last longer with code offload. In *Proceedings of the 8th international conference on Mobile systems, applications, and services*, pages 49–62. ACM, 2010.

[35] Mark S Gordon, Davoud Anoushe Jamshidi, Scott A Mahlke, Zhuoqing Morley Mao, and Xu Chen. Comet: Code offload by migrating execution transparently.

[36] Moo-Ryong Ra, Anmol Sheth, Lily Mummert, Padmanabhan Pillai, David Wetherall, and Ramesh Govindan. Odessa: enabling interactive perception applications on mobile devices. In *Proceedings of the 9th international conference on Mobile systems, applications, and services*, pages 43–56. ACM, 2011.

[8] 百度超级计算机. https://gigaom.com/2015/01/14/baidu-has-built-a-supercomputer-for-deep-learning/。访问时间：2016 年 8 月。 [9] Johann Hauswald、Yiping Kang、Michael A. Laurenzano、Quan Chen、Cheng Li、Trevor Mudge、Ronald G. Dreslinski、Jason Mars 和 Lingjia Tang。 Djinn 和 tonic：Dnn 作为一种服务及其对未来仓库规模计算机的影响。
*Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*，ISCA '15，美国纽约州纽约市，2015 年。ACM。 [10] “谷歌大脑”是真实存在的，但很少有人见过它。
http://www.businessinsider.com/what-is-google-brain-2016-9。访问时间：2017 年 1 月。 [11] 谷歌利用 TPU 定制芯片增强机器学习任务。
https://cloudplatform.googleblog.com/2016/05/Google-supercharges-machine-learning-tasks-with-custom-chip.html。访问时间：2017 年 1 月。 [12] Apple 的大型新数据中心将托管 Nuance Tech。
http://techcrunch.com/2011/05/09/apple-nuance-data-center-deal/。访问时间：2016 年 8 月。 [13] Apple 转向基于开源 Mesos 平台构建的第三代 Siri 后端。
http://9to5mac.com/2015/04/ 27/siri-backend-mesos/。访问时间：2016 年 8 月。 [14] 马修·哈尔彭、朱宇豪、维杰·贾纳帕·雷迪。移动 CPU 的崛起：量化各代移动 CPU 设计趋势对性能、能源和用户满意度的影响。在
*High Performance Computer Architecture (HPCA), 2016 IEEE International Symposium on*，第 64-76 页。 IEEE，2016。[15] 白皮书：NVIDIA Tegra X1。技术报告。访问时间：2017 年 1 月。 [16] NVIDIA Jetson TK1 开发套件：将 GPU 加速计算引入嵌入式系统。技术报告。访问时间：2017 年 1 月。 [17] Nvidia 的 Tegra K1 是 Google Nexus 9 的核心。http://www.pcmag.com/article2/0,2817,2470740,00.asp。访问时间：2016 年 8 月。 [18] 贾扬清、Evan Shelhamer、Jeff Donahue、Sergey Karayev、Jonathan Long、Ross Girshick、Sergio Guadarrama 和 Trevor Darrell。 Caffe：用于快速特征嵌入的卷积架构。 *arXiv preprint arXiv:1408.5093*，2014。[19]王茜，张贤一，张云泉，易庆。 Augem：在 x86 CPU 上自动生成高性能密集线性代数内核。 *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*，第 25 页。ACM，2013。[20] Sharan Chetlur、Cliff Woolley、Philippe Vandermersch、Jonathan Cohen、John Tran、Bryan Catanzaro 和 Evan Shelhamer。 cuDNN：深度学习的有效原语。 *CoRR*，abs/1410.0759，2014。[21] Alex Krizhevsky、Ilya Sutskever 和 Geoffrey E Hinton。使用深度卷积神经网络进行 Imagenet 分类。
*Advances in neural information processing systems*，2012 年。[22] Adam Coates、Brody Huval、Tao Wang、David Wu、Bryan Catanzaro 和 Andrew Ng。使用 cots HPC 进行深度学习系统。 *Proceedings of the 30th international conference on machine learning*，第 1337–1345 页，2013 年。 [23] TestMyNet：互联网速度测试。 http://testmy.net/。访问时间：2015 年 2 月。 [24] 瓦特增加了吗？功率计。
https://www.wattsupmeters.com/。访问时间：2015 年 5 月。 [25] 黄俊贤，钱峰，亚历山大·格伯，Z·莫利·毛，苏巴布拉塔·森，奥利弗·斯帕茨切克。仔细检查 4g lte 网络的性能和功率特性。在
*Proceedings of the 10th international conference on Mobile systems, applications, and services*，第 225-238 页。 ACM，2012。[26]凯伦·西蒙扬和安德鲁·齐瑟曼。用于大规模图像识别的非常深的卷积网络。 *arXiv preprint arXiv:1409.1556*，2014。 [27] Yaniv Taigman、Ming Yang、Marc’Aurelio Ranzato 和 Lior Wolf。 Deepface：缩小人脸验证方面与人类水平性能的差距。 *Computer Vision and Pattern Recogni- tion (CVPR)*，2014 年。[28] Yann LeCun、Léon Bottou、Yoshua Bengio 和 Patrick Haffner。基于梯度的学习应用于文档识别。
*Proceedings of the IEEE*，1998。[29] Daniel Povey、Arnab Ghoshal、Gilles Boulianne、Lukas Burget、Ondrej Glembek、Nagendra Goel、Mirko Hannemann、Petr Motlicek、Yanmin Qi、Petr Schwarz 等。 kaldi语音识别工具包。 *Proc. ASRU*，2011。[30] Ronan Collobert、Jason Weston、Léon Bottou、Michael Karlen、Koray Kavukcuoglu 和 Pavel Kuksa。自然语言处理（几乎）从头开始。 *The Journal of Ma- chine Learning Research*，2011。 [31] Ashkan Nikravesh、David R Choffnes、Ethan Katz-Bassett、Z Morley Mao 和 Matt Welsh。用户设备的移动网络性能：纵向多维分析。在
*International Conference on Passive and Active Network Measurement*，第 12-22 页。 Springer，2014。[32] David Lo、Liqun Cheng、Rama Govindaraju、Luiz André Barroso 和 Christos Kozyrakis。实现大规模延迟关键工作负载的能源比例。在 *ACM SIGARCH Computer Architecture News*，第 42 卷，第 301-312 页。 IEEE 出版社，2014 年。[33]Mark Slee、Aditya Agarwal 和 Marc Kwiatkowski。 Thrift：可扩展的跨语言服务实现。 *Facebook White Paper*，5(8)，2007。 [34] Eduardo Cuervo、Aruna Balasubramanian、Dae-ki Cho、Alec Wolman、Stefan Saroiu、Ranveer Chandra 和 Paramvir Bahl。 Maui：通过代码卸载让智能手机的使用寿命更长。在
*Proceedings of the 8th international conference on Mobile systems, applications, and services*，第 49-62 页。 ACM，2010。 [35] Mark S Gordon、Davoud Anoushe Jamshidi、Scott A Mahlke、Zhuoqing Morley Mao 和 Xu Chen。 Comet：通过透明地迁移执行来卸载代码。 [36] Moo-Ryong Ra、Anmol Sheth、Lily Mummert、Padmanabhan Pillai、David Wetherall 和 Ramesh Govindan。 Odessa：在移动设备上启用交互式感知应用程序。在 *Proceedings of the 9th international conference on Mobile systems, applications, and services*，第 43-56 页。美国CM，2011。

[37] Byung-Gon Chun, Sunghwan Ihm, Petros Maniatis, Mayur Naik, and Ashwin Patti. Clonecloud: elastic execution between mobile device and cloud. In *Proceedings of the sixth conference on Computer systems*, pages 301–314. ACM, 2011.

[38] David Meisner, Junjie Wu, and Thomas F. Wenisch. BigHouse: A Simulation Infrastructure for Data Center Systems. *ISPASS '12: International Symposium on Performance Analysis of Systems and Software*, April 2012.

[39] Chang-Hong Hsu, Yunqi Zhang, Michael A. Laurenzano, David Meisner, Thomas Wenisch, Lingjia Tang, Jason Mars, and Ronald G. Dreslinski. Adrenaline: Pinpointing and reigning in tail queries with quick voltage boosting. In *International Symposium on High Performance Computer Architecture (HPCA)*, 2015.

[40] Michael A. Laurenzano, Yunqi Zhang, Lingjia Tang, and Jason Mars. Protean code: Achieving near-free online code transformations for warehouse scale computers. In *International Symposium on Microarchitecture (MICRO)*, 2014.

[41] Jason Mars, Lingjia Tang, Robert Hundt, Kevin Skadron, and Mary Lou Soffa. Bubble-up: Increasing utilization in modern warehouse scale computers via sensible co-locations. In *International Symposium on Microarchitecture (MICRO)*, 2011.

[42] Vinicius Petrucci, Michael A. Laurenzano, Yunqi Zhang, John Doherty, Daniel Mosse, Jason Mars, and Lingjia Tang. Octopus-man: Qos-driven task management for heterogeneous multicore in warehouse scale computers. In *International Symposium on High Performance Computer Architecture (HPCA)*, 2015.

[43] Jason Mars and Lingjia Tang. Whare-map: Heterogeneity in "homogeneous" warehouse-scale computers. In *International Symposium on Computer Architecture (ISCA)*, 2013.

[44] Johann Hauswald, Tom Manville, Qi Zheng, Ronald G. Dreslinski, Chaitali Chakrabarti, and Trevor Mudge. A hybrid approach to offloading mobile image classification. In *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014.

[45] Johann Hauswald, Michael A. Laurenzano, Yunqi Zhang, Cheng Li, Austin Rovinski, Arjun Khurana, Ronald G. Dreslinski, Trevor Mudge, Vinicius Petrucci, Lingjia Tang, and Jason Mars. Sirius: An open end-to-end voice and vision personal assistant and its implications for future warehouse scale computers. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2015.

[46] Hailong Yang, Alex Breslow, Jason Mars, and Lingjia Tang. Bubble-flux: Precise online qos management for increased utilization in warehouse scale computers. In *International Symposium on Computer Architecture (ISCA)*, 2013.

[47] Matt Skach, Manish Arora, Chang-Hong Hsu, Qi Li, Dean Tullsen, Lingjia Tang, and Jason Mars. Thermal time shifting: Leveraging phase change materials to reduce cooling costs in warehouse-scale computers. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, ISCA '15, 2015.

[48] Yunqi Zhang, Michael A. Laurenzano, Jason Mars, and Lingjia Tang. Smite: Precise qos prediction on real system smt processors to improve utilization in warehouse scale computers. In *International Symposium on Microarchitecture (MICRO)*, 2014.

[49] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. Baymax: Qos awareness and increased utilization for non-preemptive accelerators in warehouse scale computers. In *ACM SIGPLAN Notices*, volume 51, pages 681–696. ACM, 2016.

[50] Yunqi Zhang, David Meisner, Jason Mars, and Lingjia Tang. Treadmill: Attributing the source of tail latency through precise load testing and statistical inference. In *Computer Architecture (ISCA), 2016 ACM/IEEE 43rd Annual International Symposium on*, pages 456–468. IEEE, 2016.

[51] Animesh Jain, Michael A Laurenzano, Lingjia Tang, and Jason Mars. Continuous shape shifting: Enabling loop co-optimization via near-free dynamic code rewriting. In *Microarchitecture (MICRO), 2016 49th Annual IEEE/ACM International Symposium on*, pages 1–12. IEEE, 2016.

[52] Michael A. Laurenzano, Yunqi Zhang, Jiang Chen, Lingjia Tang, and Jason Mars. Powerchop: Identifying and managing non-critical units in hybrid processor architectures. In *Proceedings of the 43rd International Symposium on Computer Architecture*, ISCA '16, pages 140–152, Piscataway, NJ, USA, 2016. IEEE Press.

[53] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*, pages 269–284. ACM, 2014.

[54] Daofu Liu, Tianshi Chen, Shaoli Liu, Jinhong Zhou, Shengyuan Zhou, Olivier Teman, Xiaobing Feng, Xuehai Zhou, and Yunji Chen. Pudiannao: A polyvalent machine learning accelerator. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 369–381. ACM, 2015.

[55] Kalin Ovtcharov, Olatunji Ruwase, Joo-Young Kim, Jeremy Fowers, Karin Strauss, and Eric S Chung. Accelerating deep convolutional neural networks using specialized hardware. *Microsoft Research Whitepaper*, 2(11), 2015.

[56] Xin Lei, Andrew Senior, Alexander Gruenstein, and Jeffrey Sorensen. Accurate and Compact Large vocabulary speech recognition on mobile devices. In *INTERSPEECH*, pages 662–665, 2013.

[57] Xin Lei, Andrew Senior, Alexander Gruenstein, and Jeffrey Sorensen. Accurate and compact large vocabulary speech recognition on mobile devices. In *INTERSPEECH*, pages 662–665, 2013.

[58] Seungyeop Han, Haichen Shen, Matthai Philipose, Sharad Agarwal, Alec Wolman, and Arvind Krishnamurthy. Mcdnn: An execution framework for deep neural networks on resource-constrained devices. In *MobiSys*, 2016.

[37]Byung-Gon Chun、Sunghwan Ihm、Petros Maniatis、Mayur Naik 和 Ashwin Patti。 Clonecloud：移动设备和云之间的弹性执行。在 *Proceedings of the sixth conference on Computer systems*，第 301-314 页。 ACM，2011。 [38] David Meisner、Junjie Wu 和 Thomas F. Wenisch。BigHouse：数据中心系统的模拟基础设施。*ISPASS '12: International Symposium on Performance Analysis of Systems and Software*，2012 年 4 月。[39] 徐长红、张云奇、Michael A. Laurenzano、David Meisner、Thomas Wenisch、Lingjia Tang、Jason Mars 和 Ronald G. Dreslinski。肾上腺素：通过快速升压精确定位并控制尾部查询。 *International Symposium on High Performance Computer Architecture (HPCA)*，2015 年。[40] Michael A. Laurenzano、Yunqi Zhang、Lingjia Tang 和 Jason Mars。 Protean 代码：为仓库规模的计算机实现近乎免费的在线代码转换。 *International Symposium on Microarchitecture (MICRO)*，2014 年。[41] Jason Mars、Lingjia Tang、Robert Hundt、Kevin Skadron 和 Mary Lou Soffa。泡沫化：通过合理的托管提高现代仓库规模计算机的利用率。 *International Symposium on Microarchitecture (MICRO)*，2011 年。[42] Vinicius Petrucci、Michael A. Laurenzano、Yunqi Zhang、John Doherty、Daniel Mosse、Jason Mars 和 Lingjia Tang。 Octopus-man：仓库规模计算机中异构多核的 Qos 驱动任务管理。见*International Symposium on High Performance Computer Architecture (HPCA)*，2015。 [43] Jason Mars 和 Lingjia Tang。 Wware-map：“同质”仓库规模计算机中的异构性。 *International Symposium on Computer Architecture (ISCA)*，2013 年。[44] Johann Hauswald、Tom Manville、Qi Cheng、Ronald G. Dreslinski、Chaitali Chakrabarti 和 Trevor Mudge。一种卸载移动图像分类的混合方法。 *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*，2014 年。[45] Johann Hauswald、Michael A. Laurenzano、Yunqi Zhang、Cheng Li、Austin Rovinski、Arjun Khurana、Ronald G. Dreslinski、Trevor Mudge、Vinicius Petrucci、Lingjia Tang 和 Jason Mars。 Sirius：开放式端到端语音和视觉个人助理及其对未来仓库规模计算机的影响。 *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*，2015 年。[46] 杨海龙、Alex Breslow、Jason Mars 和 Lingjia Tang。 Bubble-flux：精确的在线服务质量管理，提高仓库规模计算机的利用率。 *International Symposium on Computer Architecture (ISCA)*，2013 年。[47] Matt Skach、Manish Arora、Chang-Hong Hsu、Qi Li、Dean Tullsen、Lingjia Tang 和 Jason Mars。热时移：利用相变材料降低仓库规模计算机的冷却成本。在 *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*，ISCA '15，2015 年。

[48] 张云奇，迈克尔·A·劳伦扎诺，杰森·马尔斯，唐凌嘉。 Smite：对真实系统 smt 处理器进行精确的服务质量预测，以提高仓库规模计算机的利用率。*International Symposium on Microarchitecture (MI- CRO)*，2014年。[49]陈泉，杨海龙，杰森·马尔斯，唐凌嘉。 Baymax：仓库规模计算机中非抢占式加速器的 Qos 意识和利用率提高。在 *ACM SIGPLAN Notices*，第 51 卷，第 681-696 页。 ACM，2016。[50] 张云奇、大卫·迈斯纳、杰森·马尔斯和唐凌嘉。跑步机：通过精确的负载测试和统计推断来归因尾部延迟的来源。在 *Computer Architecture (ISCA), 2016 ACM/IEEE 43rd Annual International Symposium on*，第 456-468 页。 IEEE，2016。[51] Animesh Jain、Michael A Laurenzano、Lingjia Tang 和 Jason Mars。连续形状变换：通过近乎自由的动态代码重写实现循环协同优化。在 *Microarchitecture (MICRO), 2016 49th Annual IEEE/ACM International Symposium on*，第 1-12 页。 IEEE，2016。 [52] Michael A. Laurenzano、Yunqi 张、Jiang Chen、Lingjia Tang 和 Jason Mars。 Powerchop：识别和管理混合处理器架构中的非关键单元。*Proceedings of the 43rd International Symposium on Computer Architecture*，ISCA '16，第 140-152 页，美国新泽西州皮斯卡塔韦，2016 年。IEEE Press。 [53] 陈天石，杜子东，孙宁辉，王佳，吴成勇，陈云吉，Olivier Temam。 Diannao：用于无处不在的机器学习的小型高吞吐量加速器。在 *Proceedings of the 19th international conference on Architectural support for programming languages and operating systems*，第 269-284 页。 ACM，2014。 [54]刘道夫，陈天石，刘少力，周金红，周胜远，Olivier Teman，冯小兵，周学海，陈云吉。 Pudiannao：多价机器学习加速器。在 *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*，第 369-381 页。 ACM，2015。 [55] Kalin Ovtcharov、Olatunji Ruwase、Joo-Young Kim、Jeremy Fowers、Karin Strauss 和 Eric S Chung。使用专用硬件加速深度卷积神经网络。 *Microsoft Research Whitepaper*，2(11)，2015。[56]Xin Lei、Andrew Senior、Alexander Gruenstein 和 Jeffrey Sorensen。移动设备上准确且紧凑的大词汇量语音识别。见 *INTERSPEECH*，第 662-665 页，2013 年。 [57] Xin Lei、Andrew Senior、Alexander Gruenstein 和 Jeffrey Sorensen。移动设备上准确、紧凑的大词汇量语音识别。见 *INTERSPEECH*，第 662-665 页，2013 年。 [58] Seungyeop Han、Haichen Shen、Matthai Philipose、Sharad Agarwal、Alec Wolman 和 Arvind Krishnamurthy。 Mcdnn：资源受限设备上深度神经网络的执行框架。在 *MobiSys*，2016 年。